



Universidad Nacional
de Educación a Distancia

Escuela Técnica Superior de
Ingeniería Informática

Universidad
Complutense de Madrid

Facultad de
Informática

[Título del trabajo]

Martin Augusto Reigadas Teran

Director/a: [Director/a]
Co-director/a: [Co-director/a]

Trabajo de Fin de Máster

Máster Universitario
en Ingeniería de Control y Sistema

[Fecha de presentación/Convocatoria]

Agradecimientos

[Quisiera dedicar ...]

Resumen

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Palabras clave: palabra1, palabra2, palabra3

Abstract

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Keywords: keyword1, keyword2, keyword3

Índice general

1. Introducción **1**
1.1. Motivación 1
1.2. Planteamiento y alcance 2
1.3. Objetivos 3
1.4. Estructura del documento 3

2. Fundamentos teóricos **5**
2.1. Aprendizaje por Refuerzo 5
2.2. Aprendizaje por Refuerzo Profundo 9
2.2.1. Redes Neuronales Artificiales como Aproximadores de Funciones . . . 9
2.2.2. Taxonomía y Algoritmos Típicos de DRL 10
2.3. Cinemática Típica de un AGV 11
2.3.1. Modelo de Tracción Diferencial 11
2.3.2. Modelo de Dirección de Ackermann (Modelo de Bicicleta) 12
2.4. Cinemática Utilizada 12

3. Estado del arte **15**
3.1. Navegación móvil y planteamiento del problema 15
3.2. Métodos clásicos de navegación y enfoques aprendidos 16
3.3. Algoritmos de aprendizaje por refuerzo profundo en navegación 17
3.4. Observación local y observabilidad parcial 18
3.5. Diseño de recompensa, currículo y seguridad 19
3.6. Síntesis y relación con el trabajo 20

4. Metodología de navegación reactiva de un AGV mediante aprendizaje por refuerzo profundo **21**
4.1. Definición del problema 21
4.2. Formulación del problema como POMDP 22
4.3. Arquitectura de control 23
4.4. Modelo cinemático del AGV sobre rejilla 25
4.4.1. Modelo con giro y avance (`GridRobot`) 25

4.4.2. Modelo sin giro con desplazamiento cardinal (`GridRobotNotTurning`) 25
4.4.3. Elección del modelo cinemático . . . 26
4.5. Espacio de observaciones . . . 27
4.5.1. Sensor LiDAR por secciones angulares . . . 27
4.5.2. Variables relativas al objetivo . . . 28
4.5.3. Vector de observación . . . 28
4.6. Función de recompensa . . . 28
4.7. Arquitecturas de red neuronal . . . 30
4.7.1. Aproximador de valor de acción (DQN) . . . 30
4.7.2. Actor-crítico con PPO . . . 31
4.8. Aprendizaje por currículo (opcional) . . . 32
4.9. Enmascarado de acciones inválidas (opcional) . . . 32
4.10. Extensión recurrente (opcional) . . . 32

5. Configuración experimental y resultados **35**
5.1. Configuración experimental . . . 35
5.1.1. Escenarios de la rejilla . . . 35
5.1.2. Hiperparámetros del agente PPO . . . 36
5.1.3. Hiperparámetros del agente DQN . . . 36
5.1.4. Función de recompensa . . . 36
5.1.5. Configuración del sensor LiDAR . . . 36
5.1.6. Entorno de ejecución . . . 38
5.1.7. Protocolo de evaluación . . . 38
5.2. Resultados del baseline PPO (test 25) . . . 39
5.3. Estudios complementarios . . . 40
5.3.1. Aprendizaje por currículo (test 26) . . . 40
5.3.2. Progreso geodésico (test 27) . . . 40
5.3.3. Política recurrente con GRU (tests 28 y 29) . . . 41
5.3.4. Alcance extendido del LiDAR (test 30) . . . 41
5.3.5. Enmascarado de acciones inválidas (test 31) . . . 41
5.4. Discusión . . . 41

6. Conclusiones y trabajos futuros **43**

Bibliografía y referencias **44**

A. Detalles de implementación **47**
A.1. Gestión de interrupciones del entrenamiento . . . 47
A.2. Parada temprana por paciencia . . . 48
A.3. Muestreo del *buffer* de experiencia en PPO . . . 48

A.4. Puntos de control y recuperación 48
A.5. Métricas registradas en TensorBoard 49
A.6. Convención de nomenclatura de experimentos 49
A.7. Entorno de ejecución y dependencias 50

B. Ejemplos en LaTeX **51**
B.1. Referencias, citas y bibliografía 51
B.1.1. Referencias 51
B.1.2. Bibliografía 51
B.2. Figuras 52
B.3. Tablas 52
B.4. Código 53
B.5. Listas 54
B.6. Ecuaciones 54

Índice de figuras

2.1. Ciclo clásico de interacción Agente-Entorno en el marco del Aprendizaje por Refuerzo (RL). 6
2.2. Esquema cinemático clásico planar de un robot móvil (vista superior) para modelado físico continuo. 13

4.1. Elementos del POMDP sobre la rejilla 23
4.2. Arquitectura de control DRL 24
4.3. Acciones del modelo con giro 26
4.4. Acciones del modelo sin giro 27

B.1. Ejemplo de figura 52

Índice de tablas

4.1. Arquitecturas de red neuronal implementadas y utilizadas por los agentes DQN y PPO. La red llamada `SimpleQNetwork` se emplea tanto como aproximador $Q(s, a)$ en DQN como para producir los logits de la política en PPO; la red `ValueNetwork` actúa como crítico $V(s)$ en PPO. Las dimensiones aquí reflejan el baseline experimental; todas las redes son paramétricas en profundidad y anchura. 30

5.1. Mapas de ocupación empleados en los experimentos. Las celdas ocupadas incluyen tanto los obstáculos declarados como su posible solapamiento. 35
5.2. Hiperparámetros del agente PPO en el experimento de referencia (test 25). . 36
5.3. Hiperparámetros del agente DQN (`GridAgent`). 37
5.4. Parámetros de la función de recompensa `ProgressReward` en el experimento de referencia. 37
5.5. Resultados del agente PPO de referencia (test 25) en la evaluación final sobre los cuatro mapas (1 000 episodios por modalidad, semilla 0). Los *loops* se cuentan sobre los episodios que agotan el horizonte temporal. 39
5.6. Tasa de éxito por mapa del baseline PPO (test 25). Entre paréntesis, el número de episodios con el mapa seleccionado sobre el total de 1 000. 39
5.7. Comparación de los experimentos complementarios frente al baseline PPO (test 25). Las columnas z reportan el estadístico del test de proporciones frente al baseline sobre 1 000 episodios por modalidad; $|z| > 1{,}96$ equivale a significación al 5 %. 40

A.1. Métricas registradas por el `BaseTrainer` en cada episodio de entrenamiento. 49

B.1. Ejemplo de tabla 53

Capítulo 1

Introducción

1.1. Motivación

La automatización de los flujos internos de materiales es un elemento central de la intralogística contemporánea. En este contexto, los vehículos de guiado automático (AGV, por sus siglas en inglés) permiten transportar cargas entre distintas zonas de una planta o un almacén y pueden contribuir a flexibilizar las operaciones de fabricación y almacenamiento [Hu et al., 2020, Ye et al., 2023]. Su despliegue, sin embargo, exige resolver de manera fiable tres problemas acoplados: la localización del vehículo, la planificación de trayectorias y la evitación de los obstáculos que aparecen en el entorno. La literatura reciente aborda, por un lado, la navegación de un único AGV frente a obstáculos y, por otro, la coordinación y la planificación de rutas de varios vehículos; se trata de problemas relacionados, pero no equivalentes [Ye et al., 2023, Hu et al., 2020].

Las arquitecturas convencionales de navegación suelen combinar una representación del entorno con un planificador global y un controlador local. Este enfoque funciona razonablemente bien cuando el mapa es conocido y las condiciones de operación se mantienen estables. Cuando cambia la disposición del entorno, el mapa y las rutas deben actualizarse; además, la calidad final de la navegación depende de la calidad de la percepción y del modelo que emplea el planificador. Los planificadores globales basados en grafos o en rejillas, como A^* o sus variantes, exigen replanificar cada vez que el mapa cambia, lo que resulta costoso en entornos dinámicos [Sivashangaran and Eskandarian, 2023]. Los métodos reactivos, como los campos potenciales artificiales (APF), proponen una alternativa de bajo coste computacional: la meta genera una fuerza atractiva y los obstáculos generan fuerzas repulsivas, y el robot se mueve siguiendo la resultante. El esquema es sencillo y rápido, pero adolece de un problema bien conocido: en ciertas configuraciones geométricas la fuerza atractiva y la repulsiva se compensan, el robot queda detenido en un mínimo local y no alcanza la meta [Pérez et al., 2022]. Estas limitaciones motivan el estudio de controladores capaces de seleccionar las acciones directamente a partir de observaciones locales, sin pasar por un planificador global.

El aprendizaje por refuerzo (RL, por sus siglas en inglés) ofrece una formulación distinta de la navegación: el agente interactúa con el entorno, observa su estado, elige una acción y recibe una señal de recompensa; a partir de esta interacción aprende una política que maximiza la recompensa acumulada esperada [Shakya et al., 2023, Murphy, 2025]. Al combinar RL con redes neuronales profundas (DRL, *Deep Reinforcement Learning*) es posible aproximar políticas o funciones de valor sobre observaciones de alta dimensión. En navegación sin mapa previo (*map-less*) se han estudiado políticas que toman decisiones a partir de medidas locales de rango y de la posición relativa del objetivo, junto con estrategias de entrenamiento que facilitan la adquisición de la tarea sobre escenarios de dificultad creciente [Sivashangaran and Eskandarian, 2023, Xue et al., 2022, Guo et al., 2025]. Este enfoque no elimina la necesidad de diseñar las observaciones, la señal de recompensa y las condiciones de entrenamiento, ni garantiza por sí mismo seguridad formal o generalización a escenarios no vistos [Gao et al., 2025].

1.2. Planteamiento y alcance

Este trabajo estudia el control reactivo de un robot móvil tipo AGV en un entorno bidimensional representado mediante una rejilla con obstáculos estáticos. El agente no recibe el mapa completo como entrada de la política; emplea una observación local compuesta por medidas de distancia agrupadas en sectores angulares (procedentes de un sensor LiDAR simulado) y por información relativa al objetivo (distancia y rumbo). El espacio de acciones es discreto y, según el modelo cinemático, se reduce a avanzar y girar ($|\mathcal{A}| = 3$) o a desplazamientos cardinales ($|\mathcal{A}| = 4$).

En consecuencia, el problema abordado se plantea como una simulación de navegación en rejilla, y en ella se evalúa el comportamiento aprendido con dos algoritmos representativos: una red Q profunda (DQN) de familia *off-policy* basada en valor, la optimización de política próxima (PPO) de familia *on-policy* actor-crítico. Queda fuera del alcance de este trabajo el navegación de un AGV físico (*sim-to-real*), la coordinación entre varios AGV ni la navegación en presencia de obstáculos dinámicos (que se plantean como trabajos futuro). Las conclusiones de la memoria se derivan de los algoritmos, mapas, hiperparámetros y métricas evaluados.

La propuesta añade dos elementos a este marco básico. En primer lugar, una función de recompensa compuesta que suma a los términos "terminales" (éxito, colisión, tiempo agotado), un término de progreso proporcional a la reducción de la distancia a la meta, un pequeño coste por paso, una penalización continua por proximidad a los obstáculos. En segundo lugar, un mecanismo heurístico de detección de estancamiento (`is_stuck`) que observa la variabilidad reciente de la posición del AGV y, cuando el agente oscila dentro de un pasillo estrecho o permanece atrapado en un mínimo local de la rejilla, dispara un reinicio suave (*soft reset*) que eleva temporalmente el factor de exploración en lugar de interrumpir

el episodio.

1.3. Objetivos

Objetivo general. Diseñar, implementar y evaluar en simulación una política de navegación reactiva libre de mapas (*map-less*) para un robot móvil tipo AGV que opera en entornos bidimensionales de rejilla con obstáculos estáticos, basada en algoritmos de aprendizaje por refuerzo profundo.

Objetivos específicos.

- Formular el problema de navegación como un proceso de decisión de Markov parcialmente observable (POMDP), especificando con precisión los estados, las observaciones, las acciones, las transiciones y la señal de recompensa [Md et al., 2024, Gao et al., 2025].
- Definir una observación local compacta que combine las distancias a los obstáculos obtenidas por sectores de un LiDAR simulado con la distancia y el rumbo relativos al objetivo, de modo que la dimensión del estado observable se mantenga acotada al variar la resolución angular del sensor.
- Diseñar y documentar la propuesta de una función de recompensa original y el criterio heurístico de detección de estancamiento utilizados durante el entrenamiento, junto con el mecanismo de reinicio suave que se activa en los episodios bloqueados.
- Implementar los agentes DQN y PPO para el espacio de acciones discreto propuesto, utilizando perceptrones multicapa como aproximadores de la función de valor y de la política.
- Comparar el comportamiento de los dos agentes sobre un conjunto común de escenarios estáticos simulados, en términos de tasa de éxito, tasa de colisión, duración de los episodios y curvas de retorno, discutiendo las limitaciones de la evaluación realizada.

1.4. Estructura del documento

A continuación se detalla el contenido de cada capítulo de la memoria:

- En el **capítulo 1 (Introducción)** se expone la motivación del trabajo, se delimita su alcance y se enumeran los objetivos.
- En el **capítulo 2 (Fundamentos teóricos)** se presenta el marco matemático del aprendizaje por refuerzo (procesos de decisión de Markov, ecuaciones de Bellman, funciones de valor), su extensión al caso profundo (DRL), las arquitecturas de red más empleadas y los modelos cinemáticos típicos de los AGV. Al final del capítulo se justifica la cinemática discreta paso a paso que se adopta en este trabajo.

- En el **capítulo 3 (Estado del arte)** se revisa la literatura reciente sobre navegación móvil, aprendizaje por refuerzo profundo aplicado a AGV, percepción local bajo observabilidad parcial y estrategias de entrenamiento (recompensa, currículo y seguridad), y se sitúa este trabajo frente a esos antecedentes.
- En el **capítulo 4 (Diseño y arquitectura)** se describe la formulación del problema como POMDP, los dos modelos cinemáticos de rejilla implementados, la observación perceptual, la función de recompensa y las arquitecturas de red utilizadas por los agentes DQN y PPO.
- En el **capítulo 5 (Configuración experimental y análisis de resultados)** se detallan los escenarios de evaluación, los hiperparámetros de entrenamiento, el mecanismo de reinicio suave y las métricas empleadas, y se presentan los resultados obtenidos.
- En el **capítulo 6 (Conclusiones y trabajo futuro)** se resumen los principales hallazgos del trabajo y se proponen líneas de continuación.

Capítulo 2

Fundamentos teóricos

En este capítulo se presentan los fundamentos teóricos necesarios para comprender el desarrollo metodológico y la experimentación de este trabajo de fin de máster. En primer lugar, se expone el marco matemático del Aprendizaje por Refuerzo (RL) clásico, introduciendo la formulación de los Procesos de Decisión de Markov (MDP) y sus ecuaciones fundamentales de Bellman. Posteriormente, se detalla el Aprendizaje por Refuerzo Profundo (DRL) como una extensión para resolver espacios de alta dimensionalidad mediante redes neuronales artificiales, describiendo sus arquitecturas y algoritmos más destacados en la literatura. A continuación, se describen los modelos cinemáticos planos tradicionales utilizados típicamente para vehículos de guiado automático (AGV), tales como la tracción diferencial y el modelo de bicicleta de Ackermann. Finalmente, se justifica formalmente el modelo de cinemática simplificada discreta paso a paso implementado en este trabajo, detallando cómo la eliminación del tiempo de integración continua optimiza de manera drástica la velocidad de convergencia y la estabilidad del entrenamiento de los agentes reactivos.

2.1. Aprendizaje por Refuerzo

El Aprendizaje por Refuerzo (RL, por sus siglas en inglés, *Reinforcement Learning*) constituye un área de la inteligencia artificial y del aprendizaje automático enfocado en cómo un agente autónomo puede aprender a tomar decisiones secuenciales óptimas interactuando dinámicamente con un entorno estocástico mediante un proceso de ensayo y error [Shakya et al., 2023]. A diferencia de otros paradigmas de aprendizaje, en RL el agente no es guiado por un conjunto de datos etiquetados de antemano; en su lugar, aprende a asociar sus acciones con recompensas o penalizaciones escalares proporcionadas por el entorno como retroalimentación inmediata, con el fin de reforzar los comportamientos deseables y desincentivar los perjudiciales [Murphy, 2025]. Este ciclo interactivo continuo se ilustra de manera simplificada en la Figura 2.1, donde se muestra la retroalimentación clásica entre las acciones del agente y las respuestas de estado y recompensa del entorno.

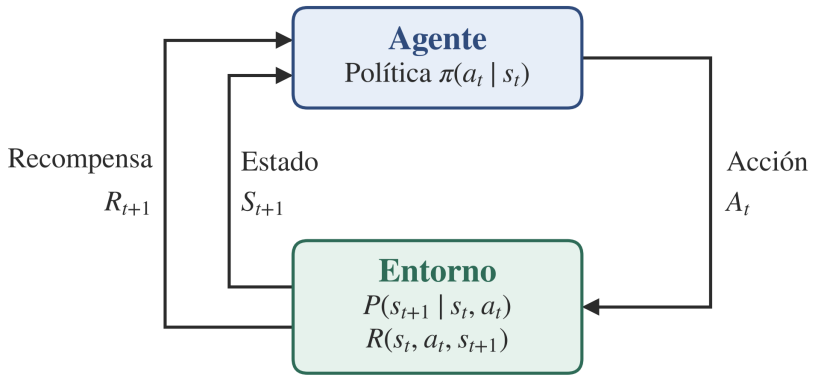


Figura 2.1: Ciclo clásico de interacción Agente-Entorno en el marco del Aprendizaje por Refuerzo (RL).

Matemáticamente, la interacción entre el agente y el entorno en pasos discretos de tiempo $t = 0, 1, 2, \ldots$ se formaliza rigurosamente bajo el marco de los Procesos de Decisión de Markov (MDP) [Murphy, 2025]. Un MDP se define formalmente como una tupla de cinco elementos $\langle S, A, P, R, \gamma \rangle$, donde [Shakya et al., 2023, Murphy, 2025]:

- S representa el espacio o conjunto de todos los estados posibles del entorno [Murphy, 2025].
- A denota el espacio de acciones disponibles para el agente [Murphy, 2025].
- $P(s_{t+1} \mid s_t, a_t)$ es la función de probabilidad de transición estocástica del estado del sistema, que define la probabilidad de transicionar al estado s_{t+1} dado que el agente ejecutó la acción a_t estando en el estado s_t [Shakya et al., 2023].
- $R(s_t, a_t, s_{t+1})$ representa la función de recompensa escalar de retroalimentación provista al agente tras realizar una transición [Shakya et al., 2023, Murphy, 2025].
- $\gamma \in [0, 1)$ es el factor de descuento temporal que pondera la ganancia futura, determinando el grado de miopía o visión a largo plazo del agente [Murphy, 2025].

Para un MDP finito, la dinámica conjunta de transición y recompensa del entorno se define mediante la función de distribución de probabilidad conjunta, expresada formalmente como [Shakya et al., 2023]:

$$p(s', r \mid s, a) \doteq \mathbb{P}(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a) \tag{2.1}$$

A partir de esta distribución conjunta, se puede deducir analíticamente la recompensa esperada del paso siguiente para cualquier par estado-acción como [Shakya et al., 2023]:

$$r_{t+1}(s, a) \doteq \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a] = \sum_{r \in \mathbb{R}} r \sum_{s' \in S} p(s', r \mid s, a) \tag{2.2}$$

Asimismo, la ganancia o retorno acumulado con descuento futuro a partir de cualquier paso de tiempo t, denotado por G_t, se define como la suma ponderada de la secuencia de recompensas futuras escalares obtenidas por el agente [Shakya et al., 2023]:

$$G_t \doteq \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \tag{2.3}$$

El retorno G_t cumple una propiedad recursiva fundamental para el desarrollo teórico y la formulación recursiva de los algoritmos de RL, la cual asocia el retorno en el instante actual con el retorno en el instante subsecuente mediante la ecuación [Shakya et al., 2023]:

$$G_t = R_{t+1} + \gamma G_{t+1} \tag{2.4}$$

El comportamiento del agente se rige por una política $\pi(a_t \mid s_t)$, la cual define una distribución de probabilidad sobre el espacio de acciones dada una observación del estado actual del sistema [Murphy, 2025]. El objetivo primordial de cualquier algoritmo de aprendizaje por refuerzo es descubrir una política óptima π^* que maximice la ganancia acumulada con descuento futuro o retorno acumulado esperado, definido formalmente como [Shakya et al., 2023]:

$$J(\pi) = \mathbb{E}_\pi[G_0] = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \right] \tag{2.5}$$

Para evaluar la calidad de una política, los algoritmos estiman funciones de valor que miden cuán beneficioso es para el agente estar en un estado particular o ejecutar una acción en dicho estado. La función de valor de estado $V^\pi(s)$ representa el retorno esperado partiendo desde el estado s y siguiendo la política π en los pasos subsecuentes [Shakya et al., 2023]:

$$V^\pi(s) = \mathbb{E}_\pi \left[G_t \mid s_t = s \right] \tag{2.6}$$

Por su parte, la función de valor de acción $Q^\pi(s, a)$ representa el retorno acumulado esperado tras ejecutar la acción a estando en el estado s, y seguir posteriormente la política π para la toma de decisiones [Shakya et al., 2023]:

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[G_t \mid s_t = s, a_t = a \right] \tag{2.7}$$

Las funciones de valor cumplen una propiedad recursiva fundamental conocida como la Ecuación de Expectación de Bellman, la cual descompone el valor de un estado como la suma de la recompensa inmediata esperada y el valor descontado de los estados sucesores [Murphy, 2025]:

$$V^\pi(s) = \sum_{a \in A} \pi(a \mid s) \sum_{s' \in S} P(s' \mid s, a) \left[R(s, a, s') + \gamma V^\pi(s') \right] \tag{2.8}$$

$$Q^{\pi}(s,a) = \sum_{s' \in S} P(s' \mid s,a) \left[R(s,a,s') + \gamma \sum_{a' \in A} \pi(a' \mid s') Q^{\pi}(s',a') \right] \tag{2.9}$$

De igual forma, las funciones de valor bajo una política óptima satisfacen las Ecuaciones de Optimalidad de Bellman, que introducen el operador de maximización rígida sobre el espacio de acciones [Shakya et al., 2023]:

$$V^{*}(s) = \max_{a \in A} \left\{ \sum_{s' \in S} P(s' \mid s,a) \left[R(s,a,s') + \gamma V^{*}(s') \right] \right\} \tag{2.10}$$

$$Q^{*}(s,a) = \sum_{s' \in S} P(s' \mid s,a) \left[R(s,a,s') + \gamma \max_{a' \in A} Q^{*}(s',a') \right] \tag{2.11}$$

Dentro de la literatura de RL, los métodos para aproximar la política óptima se dividen bajo tres taxonomías principales [Shakya et al., 2023, Murphy, 2025]:

- **Basados en Modelo (Model-based) vs. Libres de Modelo (Model-free):** Los métodos basados en modelo asumen o aprenden una estimación de las probabilidades de transición P y recompensas R del entorno para realizar planeación [Shakya et al., 2023]. En cambio, los métodos libres de modelo no requieren conocer la física o dinámica del entorno, aprendiendo de manera directa a través de las muestras de experiencia recolectadas por ensayo y error [Shakya et al., 2023].

- **En-Política (On-policy) vs. Fuera-de-Política (Off-policy):** Los métodos on-policy evalúan e incrementan de manera asíncrona la misma política utilizada para recolectar datos en la simulación [Murphy, 2025]. Por el contrario, los esquemas off-policy evalúan y optimizan una política objetivo independiente del comportamiento o la política exploratoria empleada para muestrear las transiciones, permitiendo reutilizar datos históricos de transiciones previas [Shakya et al., 2023].

- **Basados en Valor (Value-based) vs. Basados en Gradiente de Política (Policy-based):** Los esquemas basados en valor se enfocan en aproximar la función Q óptima para derivar implícitamente una política de toma de decisiones codiciosa [Murphy, 2025]. Los enfoques basados en gradiente de política optimizan de forma directa los parámetros de una red de política $\pi_{\theta}(a \mid s)$ mediante gradiente de política para maximizar los retornos acumulados de la ruta sin requerir tablas de valor explícitas [Shakya et al., 2023].

Cuando los MDPs presentan espacios de estados continuos o de dimensiones gigantescas, la representación tabular convencional de valores se vuelve inviable debido a limitaciones de memoria computacional, un problema conocido como la maldición de la dimensionalidad [Shakya et al., 2023]. Para resolver este cuello de botella técnico, surge el Aprendizaje por

Refuerzo Profundo (DRL), que fusiona el marco de decisión estocástico de RL con el poder de extracción de características jerárquicas de las redes neuronales profundas.

2.2. Aprendizaje por Refuerzo Profundo

El Aprendizaje por Refuerzo Profundo (DRL, por sus siglas en inglés, *Deep Reinforcement Learning*) revoluciona el campo del control y la navegación autónoma al incorporar Redes Neuronales Profundas (DNN) como aproximadores de funciones no lineales de alta fidelidad para representar políticas, funciones de valor o modelos de transición [Shakya et al., 2023]. Al prescindir de la selección manual de características (*hand-crafted features*) de baja dimensionalidad, las DNNs permiten al agente procesar directamente observaciones crudas y multimodales de alta dimensionalidad (tales como lecturas de LiDAR o imágenes de cámaras frontales) y mapearlas de extremo a extremo a decisiones óptimas de control [Hu et al., 2020].

2.2.1. Redes Neuronales Artificiales como Aproximadores de Funciones

Una red neuronal artificial es una estructura jerárquica de procesamiento matemático que consta de múltiples capas interconectadas de neuronas o unidades de cómputo [Shakya et al., 2023]. El tipo de red más representativo y utilizado como bloque base en DRL es el Perceptrón Multicapa o Red Totalmente Conectada (MLP, por sus siglas en inglés, *Multi-Layer Perceptron*) [Ge et al., 2024]. Un MLP típico se compone de una capa de entrada que recibe las observaciones del entorno, una o más capas ocultas intermedias responsables de la extracción progresiva de características, y una capa de salida que emite las predicciones o distribuciones de probabilidad de acciones [Hu et al., 2020].

Cada neurona de una capa oculta ejecuta una suma ponderada de sus entradas, le asocia un parámetro constante de sesgo (*bias*) e introduce no linealidad mediante una función de activación no lineal [Shakya et al., 2023]:

$$h_j = \sigma\left(\sum_i w_{ji} x_i + b_j\right) \tag{2.12}$$

Donde x_i son las señales de entrada, w_{ji} representa el peso sinóptico de la conexión, b_j es el sesgo y $\sigma(\cdot)$ es la función de activación. Las funciones de activación más comunes en el campo del DRL para introducir no linealidad en el proceso de optimización son [Shakya et al., 2023]:

- **ReLU (Rectified Linear Unit):** Definida como $\sigma(x) = \text{máx}(0, x)$, es la más utilizada en capas ocultas intermedias debido a su simplicidad computacional y su capacidad

para mitigar el desvanecimiento de gradientes.

- **Tanh (Tangente Hiperbólica):** Escala las salidas al intervalo $[-1, 1]$, siendo útil cuando las variables del estado o los rangos de las acciones requieren valores simétricos bidireccionales alrededor de cero.
- **Sigmoid:** Comprime las señales al de rango $[0, 1]$, empleada típicamente en distribuciones probabilísticas o compresión de factores de ganancia.

2.2.2. Taxonomía y Algoritmos Típicos de DRL

En función de qué componente sea aproximado por la red neuronal profunda, los algoritmos de DRL se dividen de manera clásica en dos grandes categorías, junto con arquitecturas híbridas avanzadas [Shakya et al., 2023]:

- **Algoritmos Basados en Valor (Value-based Methods):** Aproximan la función de valor de acción óptima $Q^*(s, a)$ mediante una red neuronal parametrizada por pesos θ, denotada como $Q_\theta(s, a)$ [Shakya et al., 2023].
  - *DQN (Deep Q-Network):* Incorpora dos mecanismos cruciales de estabilidad para romper la correlación temporal de las muestras y estabilizar el entrenamiento: el buffer de replay de experiencia (Experience Replay Buffer) y una red Q objetivo (Target Network) congelada temporalmente para estabilizar los objetivos de actualización de Bellman [Hu et al., 2020].
  - *Double DQN (DDQN):* Desacopla la selección de la mejor acción de su respectiva evaluación utilizando dos conjuntos distintos de pesos de red, mitigando con éxito el sesgo de sobreestimación característico del DQN estándar [Shakya et al., 2023].
  - *Dueling DQN:* Modifica la arquitectura de la red para de manera separada estimar la función de valor de estado $V(s)$ y la ventaja de acción $A(s, a)$, mejorando la precisión en estados donde muchas acciones tienen similar valor esperado [Shakya et al., 2023].
- **Algoritmos Basados en Política y Actor-Crítico (Actor-Critic Methods):** Optimizan de forma directa la política mediante gradientes y resuelven el problema de la alta varianza mediante el uso coordinado de dos aproximadores de red [Shakya et al., 2023]. El Actor representa la política $\pi_\theta(a \mid s)$ y se encarga de seleccionar acciones, mientras que el Crítico aproxima la función de valor $V_\phi(s)$ o $Q_\phi(s, a)$ y evalúa las decisiones tomadas por el actor [Sivashangaran and Eskandarian, 2023].
  - *DDPG (Deep Deterministic Policy Gradient):* Algoritmo off-policy que adapta el concepto de DQN al control de espacios de acciones continuos y de alta dimensionalidad, emitiendo comandos de actuación deterministas [Sivashangaran and Eskandarian, 2023].

- *TD3 (Twin Delayed DDPG):* Extiende DDPG introduciendo críticos gemelos minimizados para combatir la sobreestimación del valor Q, actualizando retardadamente la red del actor y suavizando las acciones objetivo con ruido de exploración regularizado [Sivashangaran and Eskandarian, 2023].
- *SAC (Soft Actor-Critic):* Algoritmo off-policy fundamentado en el Aprendizaje por Refuerzo de Máxima Entropía [Xue et al., 2022]. SAC optimiza de manera coordinada el retorno acumulado esperado y la entropía de la política, lo que incentiva de manera implícita una exploración sumamente agresiva e impide que la política colapse prematuramente en mínimos locales subóptimas [Xue et al., 2022, Sivashangaran and Eskandarian, 2023].
- *PPO (Proximal Policy Optimization):* Algoritmo on-policy que destaca por su extraordinaria estabilidad matemática [Ge et al., 2024]. PPO implementa una función de pérdida sustituta recortada (*clipped surrogate objective*) que limita de forma estricta la magnitud de los cambios en la actualización de la política entre iteraciones sucesivas, evitando colapsos catastróficos de aprendizaje sin requerir la complejidad computacional de las restricciones de segundo orden [Ge et al., 2024].

2.3. Cinemática Típica de un AGV

El diseño y despliegue de de sistemas de navegación autónoma reactiva exige comprender el comportamiento mecánico y cinemático de los Vehículos de Guiado Automático (AGV) que operan en entornos industriales y almacenes logísticos estructurados [Ye et al., 2023]. En el plano bidimensional continuo XY, la pose de un robot móvil se define comúnmente mediante el vector tridimensional de estado $\mathbf{x} = [x, y, \psi]^T$, donde (x, y) representan las coordenadas cartesianas del centro de masa, y ψ es el ángulo de rumbo u orientación angular con respecto al marco de referencia de la inercia global [Sivashangaran and Eskandarian, 2023].

En la robótica terrestre, las dos configuraciones cinemáticas continuas más representativas y estudiadas en la literatura científica son:

2.3.1. Modelo de Tracción Diferencial

Este modelo cinemático se aplica a AGVs planos que constan de dos ruedas motrices coaxiales independientes controladas por motores individuales de velocidad, junto con ruedas locas pasivas (*casters*) delanteras y traseras para conservar el balance mecánico de la plataforma [Pérez et al., 2022]. La cinemática del robot diferencial continuo se define mediante un modelo no holonómico de rodadura pura sin deslizamiento lateral, cuyas ecuaciones diferenciales de movimiento responden a la velocidad lineal instantánea longitudinal $v(t)$ y a la

velocidad angular del chasis $\omega(t)$ [Pérez et al., 2022]:

$$\dot{x}(t) = v(t)\cos(\psi(t)) \tag{2.13}$$

$$\dot{y}(t) = v(t)\sin(\psi(t)) \tag{2.14}$$

$$\dot{\psi}(t) = \omega(t) \tag{2.15}$$

Sujeto a la restricción no holonómica de no deslizamiento lateral:

$$\dot{x}(t)\sin(\psi(t)) - \dot{y}(t)\cos(\psi(t)) = 0 \tag{2.16}$$

2.3.2. Modelo de Dirección de Ackermann (Modelo de Bicicleta)

El modelo de Ackermann es típico de vehículos industriales guiados o automóviles autónomos que orientan mecánicamente su eje directriz delantero [Sivashangaran and Eskandarian, 2023]. Para simplificar el análisis y optimizar la velocidad de cómputo en el controlador del simulador, la cinemática se modela rigurosamente utilizando el modelo de bicicleta (*kinematic bicycle model*), agrupando las ruedas de cada eje en un solo punto equivalente situado en el centro de los ejes delantero y trasero [Sivashangaran and Eskandarian, 2023]. Las ecuaciones cinemáticas de actualización se definen por:

$$\dot{x}(t) = v(t)\cos(\psi(t) + \beta(t)) \tag{2.17}$$

$$\dot{y}(t) = v(t)\sin(\psi(t) + \beta(t)) \tag{2.18}$$

$$\dot{\psi}(t) = \frac{v(t)}{l_r}\sin(\beta(t)) \tag{2.19}$$

Con el ángulo de deslizamiento del centro de masa $\beta(t)$ definido por:

$$\beta(t) = \tan^{-1}\left(\frac{l_r}{l_f + l_r}\tan(\delta(t))\right) \tag{2.20}$$

Donde l_f y l_r representan las distancias desde el centro de masa del AGV hacia los ejes delantero y trasero respectivamente, $\delta(t)$ representa el ángulo de dirección de la rueda delantera de Ackermann, y $v(t)$ es la velocidad longitudinal del vehículo directriz [Sivashangaran and Eskandarian, 2023].

2.4. Cinemática Utilizada

A diferencia de los modelos cinemáticos continuos tradicionales basados en ecuaciones diferenciales acopladas y tiempos de integración infinitesimales, la presente tesis de maestría adopta y fundamenta analíticamente una cinemática simplificada y puramente discreta de

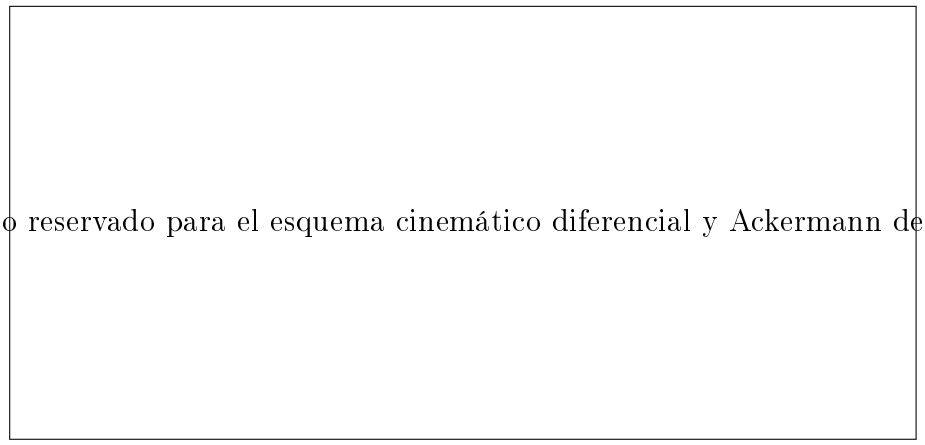


Figura 2.2: Esquema cinemático clásico planar de un robot móvil (vista superior) para modelado físico continuo.

paso de iteración. En este modelo de simulación interactiva, el concepto clásico de velocidad física continua $v(t)$, aceleración lineal $a(t)$ o paso de tiempo infinitesimal diferencial Δt desaparece por completo [Reigadas Terán, 2026a]. El tiempo del sistema se modela estrictamente como una sucesión discreta de iteraciones o pasos de simulación del entorno de aprendizaje $t \rightarrow t + 1$.

La justificación teórica y práctica de este diseño metodológico simplificado responde a tres pilares esenciales:

1. **Aceleración Exponencial del Aprendizaje (Convergencia de DRL):** La simulación e integración de físicas continuas acopladas requiere de un cómputo masivo que ralentiza de forma severa el bucle de simulación, incrementando los tiempos de entrenamiento a varios días o semanas en CPU/GPU [Sivashangaran and Eskandarian, 2023]. Al discretizar y unificar cada movimiento como una transición rígida unitaria (un paso de celda por iteración), el espacio de búsqueda del optimizador de DRL disminuye drásticamente, lo cual acelera la convergencia de las políticas reactivas en una fracción del tiempo requerido por métodos continuos tradicionales [Xue et al., 2022].

2. **Estabilidad Absoluta y Evasión de Derivas Numéricas:** En escenarios industriales dinámicos con pasillos sumamente estrechos y curvas ortogonales, los modelos continuos con inercia rotacional y fuerzas de tracción reales son propensos a experimentar deslizamientos o derivas acumuladas debido al ruido de los motores. Estas imprecisiones físicas introducen inestabilidad en la actualización de las redes Q [Xue et al., 2022]. El modelo discreto garantiza transiciones cinemáticas perfectas celda por celda sin inercia mecánica, facilitando al agente el aprendizaje geométrico de rutas colisión-free en rejillas regulares de ocupación (*GridMap*) [Reigadas Terán, 2026a].

3. **Consistencia y unificación con la Rejilla de Ocupación:** Al operar el AGV sobre un entorno discretizado por cuadrículas, la pose del robot se adapta naturalmente al

espacio entero $\mathbb{Z}^2$. De esta manera, el procesamiento local de las distancias radiales del sensor LiDAR (`GridLidar`) y la navegación relativa hacia el objetivo se evalúan de forma unificada paso a paso, eliminando la necesidad de fusionar mecánicas continuas de tracción diferencial con mallas de celdas estáticas discretas [Reigadas Terán, 2026b].

Este marco teórico de cinemática discreta paso a paso constituye la base para formular matemáticamente las transiciones específicas de movimiento de los robots implementados en este trabajo (`GridRobot` y `GridRobotNotTurning`), cuyas ecuaciones discretas de transición de pose y espacios de acciones unificados de cardinalidad $|A| = 3$ y $|A| = 4$ se desarrollarán rigurosamente en el Capítulo 4 (Metodología).

Capítulo 3

Estado del arte

En este capítulo se revisan los antecedentes directamente relacionados con el problema abordado en la memoria: la navegación reactiva de un robot móvil tipo AGV en un entorno de rejilla con obstáculos estáticos y acciones discretas, utilizando aprendizaje por refuerzo profundo. La revisión se organiza en cinco bloques. El primero sitúa la navegación móvil y precisa qué se entiende por navegación sin mapa previo. El segundo repasa los métodos clásicos de planificación y los enfoques aprendidos que los sustituyen o los complementan. El tercero resume las familias de algoritmos de aprendizaje por refuerzo profundo más empleadas en navegación. El cuarto analiza las implicaciones de la observabilidad parcial y los diseños de observación local. El quinto trata el diseño de la recompensa, el aprendizaje por currículo y la navegación segura. El capítulo cierra con una síntesis que delimita la contribución de este trabajo frente a la literatura revisada.

3.1. Navegación móvil y planteamiento del problema

La navegación de un robot móvil comprende la selección de los movimientos que permiten alcanzar un destino sin colisionar con los elementos del entorno. Los sistemas basados en mapas separan habitualmente el problema en dos niveles: un planificador global que utiliza una representación del espacio para calcular una ruta hasta la meta y un controlador local que adapta el movimiento a la información sensorial inmediata. En aplicaciones intralogísticas, la planificación de rutas de varios AGV debe considerar además la asignación de recursos y la gestión de conflictos entre vehículos; se trata de un problema de coordinación multiagente, distinto del control reactivo de un único robot frente a obstáculos [Hu et al., 2020, Ye et al., 2023].

El término *map-less* (navegación sin mapa previo) describe el tipo de información que recibe el controlador en tiempo de ejecución, no una propiedad general del sistema. Un robot *map-less* toma decisiones a partir de observaciones locales (típicamente lecturas de un sensor de rango como un LiDAR y la posición relativa del objetivo) sin disponer de un mapa

global del entorno en la entrada de la política. Esto no implica que el sistema prescinda de localización, de percepción o del cuidado en el diseño del entorno de entrenamiento, ni que la política aprendida sea equivalente al pipeline clásico de SLAM más planificación global. Los trabajos sobre exploración autónoma de vehículos terrestres y sobre navegación intralogística ilustran distintas formulaciones de este enfoque, con diferencias importantes en los sensores, la dinámica, el espacio de acciones y los simuladores empleados [Sivashangaran and Eskandarian, 2023, Xue et al., 2022, Guo et al., 2025].

3.2. Métodos clásicos de navegación y enfoques aprendidos

Los planificadores globales basados en grafos (como A^*, Dijkstra o D^*) o en muestreo del espacio libre (como PGM o RRT) proporcionan rutas sistemáticas cuando se dispone de un mapa adecuado y se admite el coste de construirlo y mantenerlo. Su rendimiento depende de la representación del entorno y del coste de replanificar cuando esa representación cambia. En entornos poco estructurados o con alta variabilidad, el replanteamiento puede ser costoso y la calidad del resultado degrada rápidamente con el ruido sensorial [Sivashangaran and Eskandarian, 2023].

Los métodos reactivos calculan la acción directamente a partir de la información local, sin construir una ruta explícita. Los campos potenciales artificiales (APF) son el ejemplo arquetípico: la meta genera una fuerza atractiva y los obstáculos generan fuerzas repulsivas, y el robot se mueve siguiendo la fuerza resultante. Son sencillos, rápidos y fáciles de ajustar, pero en determinadas configuraciones geométricas (pasillos simétricos, obstáculos cóncavos, situaciones en las que la fuerza atractiva y la repulsiva se compensan) producen puntos de equilibrio en los que el robot se detiene antes de alcanzar la meta [Pérez et al., 2022]. Las variantes clásicas mitigan este comportamiento mediante perturbaciones aleatorias o fuerzas tangenciales, pero no lo eliminan.

A caballo entre ambos enfoques se encuentran los controladores basados en redes neuronales ajustadas con algoritmos de inspiración biológica. Pérez et al. [Pérez et al., 2022] comparan APF con una red neuronal cuyos pesos se optimizan mediante un algoritmo genético sobre mapas de distinta dificultad (uno vacío, uno con un obstáculo rectangular y un laberinto). La política aprendida supera al controlador APF al reducir la frecuencia con que el robot queda atrapado en mínimos locales, aunque a costa de un proceso de entrenamiento más laborioso. Este trabajo es relevante porque documenta el problema del mínimo local sobre escenarios conceptualmente análogos a los que se evalúan en esta tesis.

El aprendizaje por refuerzo propone una tercera vía: optimizar la política directamente mediante la interacción con el entorno y una señal escalar de recompensa. Al combinarlo con redes neuronales profundas se obtiene DRL, que permite trabajar con observaciones de

alta dimensión y espacios de acción tanto discretos como continuos. Las ventajas potenciales (no requiere un modelo analítico del entorno, aprende de muestras y escala a observaciones multimodales) vienen acompañadas de desafíos conocidos: la alta cantidad de interacciones necesarias para converger, la sensibilidad al diseño de la recompensa y a los hiperparámetros, y la dificultad de garantizar que la política aprendida generalice a condiciones no vistas [Shakya et al., 2023, Sivashangaran and Eskandarian, 2023]. En consecuencia, una comparación experimental entre familias de algoritmos sólo es informativa si se detalla el entorno, el presupuesto de interacción y las métricas empleadas.

3.3. Algoritmos de aprendizaje por refuerzo profundo en navegación

Los algoritmos de DRL se clasifican habitualmente según cómo representan la política y cómo reutilizan las transiciones recogidas durante la interacción [Shakya et al., 2023, Murphy, 2025]. En el contexto de la navegación de AGV, cuatro familias concentran la mayor parte de la literatura relevante.

Los métodos basados en valor, como las redes Q profundas (DQN) y sus extensiones Double DQN y Dueling DQN, aproximan la función de valor de acción óptima $Q^*(s, a)$ y derivan de ella una política codiciosa. Son métodos *off-policy*, por lo que admiten el reuso de transiciones almacenadas en un buffer de experiencia, y resultan apropiados cuando el espacio de acciones es discreto y relativamente pequeño [Hu et al., 2020]. En el ámbito de los AGV, Hu et al. [Hu et al., 2020] utilizan DQN para la planificación en tiempo real de un AGV sobre un taller flexible, combinando la política aprendida con una regla heurística para cubrir los casos excepcionales. Más recientemente, Gao et al. [Gao et al., 2025] aplican una variante de Dueling DQN con reproducción retrospectiva de experiencia (HER) como política de alto nivel en un esquema jerárquico de navegación segura basado en subobjetivos.

Los métodos de gradiente de política optimizan de manera directa los parámetros de una política estocástica $\pi_\theta(a \mid s)$. El algoritmo de optimización de política próxima (PPO) limita el tamaño efectivo de las actualizaciones mediante un objetivo recortado (*clipped surrogate objective*), lo que suele traducirse en una convergencia más estable que la de métodos de gradiente anteriores sin exigir restricciones de segundo orden. PPO es *on-policy*: cada conjunto de transiciones se utiliza durante un número acotado de actualizaciones y después se descarta. En consecuencia, es menos eficiente en el uso de muestras que las alternativas *off-policy*, lo que puede ser un inconveniente cuando el simulador es costoso de ejecutar [Sivashangaran and Eskandarian, 2023, Ge et al., 2024]. Esta tensión entre estabilidad y eficiencia de muestreo es parte de la motivación para comparar PPO con DQN en el presente trabajo.

Los métodos actor-crítico off-policy para acciones continuas (DDPG, TD3 y SAC) extienden el esquema actor-crítico al control continuo y reutilizan transiciones del buffer. DDPG

emite comandos deterministas y sufre de inestabilidad y sobreestimación del valor Q; TD3 lo corrige con dos críticos gemelos, actualizaciones retardadas del actor y suavizado del objetivo. SAC introduce un término de entropía en el objetivo, lo que fomenta la exploración y reduce la probabilidad de colapso prematuro en mínimos locales [Sivashangaran and Eskandarian, 2023, Xue et al., 2022]. Sivashangaran y Eskandarian [Sivashangaran and Eskandarian, 2023] comparan DDPG, TD3 y SAC en exploración de AGV sin mapa previo sobre dos entornos de entrenamiento y un tercero de evaluación sin reentrenamiento, con el fin de medir la generalización fuera de la distribución de entrenamiento. Estos métodos no son parte de la comparación experimental de este trabajo, cuyo espacio de acciones es discreto, pero son antecedentes necesarios para contextualizar las decisiones de diseño.

Finalmente, los enfoques jerárquicos descomponen la navegación en decisiones de distintos niveles. Gao et al. [Gao et al., 2025] proponen un marco jerárquico en el que una política de alto nivel genera subobjetivos dentro del alcance del sensor y una política de bajo nivel basada en Safe RL ejecuta movimientos locales libres de colisión; el esquema está pensado para evitar los mínimos locales en entornos interiores. La propuesta de este trabajo no es jerárquica, pero el mecanismo de reinicio suave que se describe en el capítulo de diseño comparte con los métodos jerárquicos la intención de disponer de un recurso específico para los episodios en los que la política reactiva se detiene antes de alcanzar la meta.

3.4. Observación local y observabilidad parcial

Cuando el controlador recibe únicamente medidas locales, la observación instantánea no identifica de forma unívoca la configuración global del entorno. Dos poses del robot con lecturas de LiDAR idénticas, pero distintas en el resto del mapa, aparecen como el mismo estado para la política. Este tipo de ambigüedad se modela mediante un proceso de decisión de Markov parcialmente observable (POMDP), que generaliza al MDP clásico distinguiendo entre el estado real s_t, no accesible, y la observación o_t obtenida mediante un modelo de observación estocástico [Md et al., 2024, Gao et al., 2025]. Dado que estimar la distribución exacta de creencias es intratable en el caso general, la práctica habitual en DRL consiste en construir una observación cuidadosamente diseñada o en dotar a la red de memoria.

En navegación map-less, la observación suele combinar lecturas de un sensor de rango, variables relativas al objetivo (distancia y rumbo) y, con frecuencia, variables cinemáticas o la acción previa. La agrupación angular de un LiDAR en sectores reduce drásticamente la dimensión de entrada y preserva información suficiente sobre los obstáculos cercanos, aunque sacrifica el detalle geométrico fino. Xue et al. [Xue et al., 2022] utilizan dos LiDAR, una cámara RGB frontal y una fusión multimodal con un extractor de características preentrenado para capturar simultáneamente la geometría cercana y la posición del objetivo. En el extremo opuesto, Gao et al. [Gao et al., 2025] construyen un mapa local de obstáculos discretizado en celdas, mientras que Guo et al. [Guo et al., 2025] describen políticas basadas

únicamente en el vector de lecturas de distancia y la pose relativa de la meta. La elección depende del coste de cómputo, de la riqueza del simulador y de la tarea.

Cuando la ambigüedad persiste, la literatura introduce memoria temporal. Las arquitecturas recurrentes (LSTM, GRU) procesan una ventana histórica de observaciones y acciones y extraen un estado interno equivalente a un estimador aproximado de la creencia [Mienye et al., 2024]. En el mismo sentido se han propuesto arquitecturas autorregresivas como los Decision Transformers, que modelan la secuencia completa de observaciones, acciones y retornos [Ge et al., 2024]. Estas extensiones no son automáticas: incluir memoria recurrente mejora el rendimiento cuando la ambigüedad temporal es la causa principal del fallo, pero no en todos los problemas parcialmente observables. En este trabajo, la observación se amplía con la acción previa a_{t-1} como forma minimalista de memoria; la ampliación a una política recurrente se discute como extensión en el capítulo de diseño.

3.5. Diseño de recompensa, currículo y seguridad

La función de recompensa condiciona de forma directa el comportamiento que aprende el agente. Una señal exclusivamente terminal (positiva al llegar a la meta, negativa en colisión) resulta demasiado dispersa para tareas de horizonte largo en entornos con obstáculos: la probabilidad de alcanzar la meta por azar al principio del entrenamiento es muy baja y el gradiente de la política se queda sin señal útil. Es habitual, por tanto, añadir términos densos que *dirigen* la exploración: un término de progreso proporcional a la reducción de la distancia a la meta, un pequeño coste por paso que penaliza las rutas excesivamente largas y, cuando la seguridad importa, una penalización continua por proximidad a los obstáculos. Esquemas de este tipo se emplean en los trabajos revisados sobre navegación map-less [Gao et al., 2025, Guo et al., 2025, Xue et al., 2022] y son la base de la función de recompensa compuesta que se propone en el capítulo de diseño. La escala relativa de cada término determina el compromiso entre eficiencia, seguridad y tasa de éxito, por lo que es imprescindible documentar las magnitudes empleadas.

El aprendizaje por currículo modifica la distribución de las tareas de entrenamiento, comenzando con escenarios más fáciles y aumentando gradualmente la dificultad. En navegación intralogística, Xue et al. [Xue et al., 2022] proponen *NavACL* y su versión extendida con realimentación del crítico, *NavACL-Q*, que seleccionan de forma automática las poses iniciales de entrenamiento utilizando métricas geométricas (distancia geodésica, holgura ante obstáculos, ángulo relativo al objetivo) y el valor Q estimado por el crítico; en sus experimentos, el currículo mejora el rendimiento en torno a un 40 % respecto al entrenamiento con poses aleatorias. En este trabajo no se emplea aprendizaje por currículo explícito, pero el mecanismo de reinicio suave comparte con él la idea de modular dinámicamente la distribución de estados de inicio para desatascar la política.

La navegación segura constituye una línea de investigación propia, con objetivos y méto-

dos distintos de añadir una penalización por colisión a la recompensa. En Safe RL, el riesgo se modela mediante restricciones explícitas (por ejemplo, mediante optimización de política con restricciones, CPO) o mediante arquitecturas que combinan la política aprendida con una capa de seguridad formal, como barreras de control. En este sentido, el esquema jerárquico de Gao et al. [Gao et al., 2025] dedica la política de bajo nivel a Safe RL con estimación de la congestión del entorno. Por su parte, Md et al. [Md et al., 2024] abordan el POMDP del vehículo autónomo con un enfoque de aprendizaje por refuerzo continuo (*life-long*) que acumula la experiencia de tareas sucesivas sin olvidar las anteriores. En consecuencia, una penalización continua por proximidad en la recompensa, como la que se propone en este trabajo, debe presentarse como un mecanismo para configurar el equilibrio entre seguridad y progreso durante el entrenamiento, no como una garantía formal de seguridad.

3.6. Síntesis y relación con el trabajo

La literatura revisada sitúa este trabajo en la intersección entre control reactivo, observación local y aprendizaje por refuerzo profundo. Los antecedentes muestran alternativas de planificación clásica y aprendida, distintas familias algorítmicas para acciones discretas o continuas, diseños de observación con y sin memoria temporal y estrategias para mitigar la dificultad de la exploración (currículo automático, esquemas jerárquicos, Safe RL). Las conclusiones de cada estudio dependen del robot, del simulador, de los sensores y del protocolo experimental empleados, por lo que no son directamente intercambiables.

El presente trabajo acota la pregunta a la navegación de un AGV en una rejilla bidimensional con obstáculos estáticos y espacio de acciones discreto. Sobre esta base, se compara un algoritmo *off-policy* basado en valor (DQN) con un algoritmo *on-policy* actor-crítico (PPO), se emplea una observación compuesta por lecturas de LiDAR sectorizado y variables relativas al objetivo, se adopta una función de recompensa compuesta con penalización por proximidad y se incorpora un mecanismo heurístico de detección de estancamiento con reinicio suave para los episodios bloqueados. La aportación específica se evalúa en los escenarios y con las métricas descritos en los capítulos de diseño y de resultados; no se extrapolará a navegación física, a obstáculos dinámicos, a la coordinación entre varios AGV ni a garantías formales de seguridad sin realizar experimentos específicamente diseñados para ello.

Capítulo 4

Metodología de navegación reactiva de un AGV mediante aprendizaje por refuerzo profundo

En este capítulo se describe el sistema desarrollado para resolver el problema de navegación reactiva planteado en los capítulos anteriores. Se parte de la definición del problema y de su formulación matemática como proceso de decisión parcialmente observable, se detallan la arquitectura de control y los componentes del simulador (mapa, robot, sensor LiDAR y entorno Gymnasium), se formalizan la observación local y la función de recompensa compuesta, y se presentan las arquitecturas de red y los algoritmos de aprendizaje empleados por los agentes DQN y PPO. Las ampliaciones del esquema básico (currículo automático y enmascarado de acciones inválidas) se tratan al final como mecanismos opcionales utilizados en los experimentos descritos en el capítulo siguiente. Las utilidades operativas del entrenador (gestión de interrupciones, muestreo del replay *buffer*, puntos de control) se describen en el Apéndice A.

4.1. Definición del problema

El problema que se aborda en este trabajo consiste en que un AGV se desplace de forma autónoma y segura desde una posición de partida A hasta una posición objetivo B dentro de un entorno plano con obstáculos estáticos, cuya disposición desconoce, sin colisionar en ningún momento con ellos. El vehículo no cuenta con un mapa previo del entorno, de modo que su única fuente de información sobre los obstáculos es un sensor de barrido láser (LiDAR) embarcado, de alcance limitado, que le proporciona en cada instante la distancia al obstáculo más próximo en un conjunto discreto de direcciones relativas a su propia orientación. A esta percepción local únicamente se añaden la distancia y el rumbo relativos a la meta; el AGV no recibe ninguna información sobre las zonas del entorno que quedan fuera del alcance del

sensor. Se trata, por tanto, de obtener una política de navegación reactiva que, a partir de esta información local e incompleta, decida en cada paso el movimiento a ejecutar de modo que el AGV alcance la meta en el menor número de pasos posible y sin colisionar.

Para resolver este problema, la política de navegación se aprende en simulación mediante aprendizaje por refuerzo profundo. El entorno plano se representa mediante una rejilla de ocupación y el movimiento del AGV se restringe a acciones discretas sobre ella, para las que se consideran dos modelos cinemáticos (Sección 4.4): uno con giro, en el que el vehículo avanza según su orientación y puede rotar sobre sí mismo, y otro sin giro, en el que se desplaza directamente a cualquiera de las cuatro celdas vecinas.

4.2. Formulación del problema como POMDP

Puesto que el agente no dispone del mapa de la rejilla y sólo percibe el entorno a través del LiDAR y de las variables relativas a la meta, el problema descrito en la sección anterior se modela como un proceso de decisión de Markov parcialmente observable (POMDP), definido por la tupla $\langle \mathcal{S}, \mathcal{A}, T, R, \Omega, O, \gamma \rangle$ [Md et al., 2024, Gao et al., 2025]:

- $\mathcal{S}$: espacio de estados reales, formado por el estado cinemático del robot y la configuración completa de los obstáculos del mapa activo. El estado cinemático depende del modelo de movimiento (Sección 4.4): en el modelo con giro (`GridRobot`) es la pose $\mathbf{x}_t = (c_t, r_t, \theta_t)$, que incluye la orientación del vehículo, mientras que en el modelo sin giro (`GridRobotNotTurning`) la orientación es constante y el estado se reduce a la posición $\mathbf{p}_t = (c_t, r_t)$.

- $\mathcal{A}$: espacio discreto de acciones del robot, cuya cardinalidad también depende del modelo cinemático: $|\mathcal{A}| = 3$ en el modelo con giro (avanzar, girar a la izquierda y girar a la derecha) y $|\mathcal{A}| = 4$ en el modelo sin giro (desplazamientos hacia el norte, sur, este y oeste).

- $T(s_{t+1} \mid s_t, a_t)$: función de transición de estados. En el entorno implementado la transición es determinista salvo por el muestreo del punto de inicio y de la meta en cada episodio.

- $R(s_t, a_t, s_{t+1})$: función de recompensa compuesta descrita en la Sección 4.6.

- Ω: espacio de observaciones locales accesibles al agente, formado por las lecturas del LiDAR y por la distancia y el rumbo relativos a la meta.

- $O(o_t \mid s_t, a_{t-1})$: función de observación, estocástica por el ruido gaussiano aditivo del LiDAR.

- $\gamma \in [0, 1)$: factor de descuento temporal.

La figura 4.1 ilustra estos elementos sobre la rejilla. El estado real incluye la posición del AGV (y, en el modelo con giro, su orientación) y el mapa completo, pero el agente sólo accede a la observación local: los impactos de los rayos del LiDAR y la distancia y el rumbo a la meta B. Las acciones disponibles y el modo en que cada una modifica el estado dependen del modelo cinemático, y se detallan en la Sección 4.4.

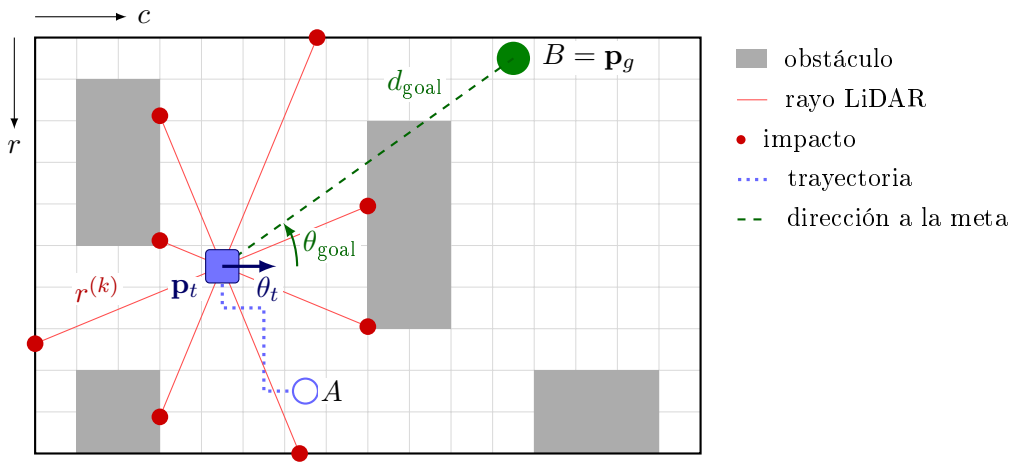


Figura 4.1: Elementos del POMDP sobre la rejilla de ocupación: estado del AGV (posición $\mathbf{p}_t$ y orientación θ_t) en un mapa con obstáculos rectangulares, trayectoria recorrida desde el punto de partida A, meta $B = \mathbf{p}_g$ y observación local, formada por la lectura más próxima del LiDAR en cada uno de los $N_{\text{sec}} = 8$ sectores ($r^{(k)}$) y por la distancia d_{goal} y el rumbo θ_{goal} a la meta, medido respecto a la orientación del robot. En el modelo sin giro la orientación es constante y el rumbo se mide siempre respecto al eje c. El agente no conoce el mapa: sólo percibe los obstáculos a través de los impactos de los rayos. Fuente: Realización propia

Como el estado real s_t no es accesible, en la literatura de POMDP se define un estado de creencia $b_t(s) \in \mathcal{B}$ sobre $\mathcal{S}$ que se actualiza recursivamente mediante [Md et al., 2024]:

$$b_{t+1}(s') = \eta\, O(o_{t+1} \mid s', a_t) \sum_{s \in \mathcal{S}} T(s' \mid s, a_t)\, b_t(s), \tag{4.1}$$

donde η es una constante de normalización. En la práctica de DRL este estado de creencia no se mantiene de forma explícita, sino que la red neuronal lo aproxima a través de una observación aumentada o, alternativamente, de una capa recurrente. En este trabajo se opta por la primera vía: la observación se compone de la lectura actual del LiDAR y de las variables relativas al objetivo (Sección 4.5). La segunda vía se ha implementado como extensión opcional (Sección 4.10) y se evalúa como estudio complementario.

4.3. Arquitectura de control

El sistema sigue el ciclo clásico del aprendizaje por refuerzo: el entorno simulado genera una observación o_t y una recompensa r_t a partir del estado del robot, el mapa y la meta;

el agente elige una acción discreta a_t en función de su política $\pi(a \mid o)$; el entorno ejecuta la acción, decide si el episodio ha terminado y entrega la nueva observación. La figura 4.2 resume la estructura y el flujo de información.

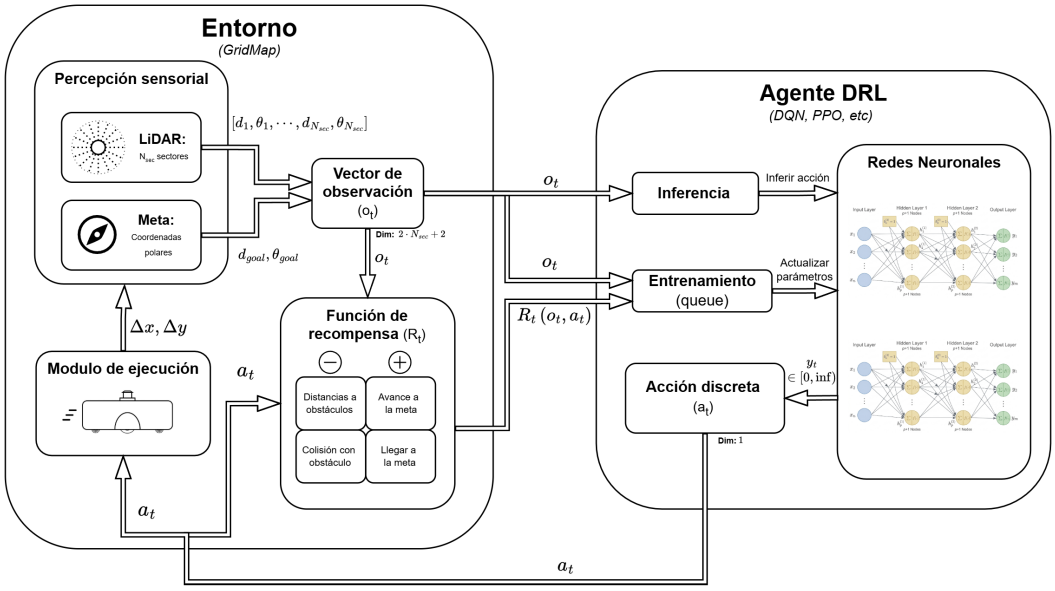


Figura 4.2: Arquitectura de control por aprendizaje por refuerzo profundo. El entorno (`BaseGridEnv`) compone mapa, robot y sensor LiDAR, calcula la señal de recompensa a partir del contrato `RewardFunction` y entrega (o_t, r_t) al agente. El agente elige a_t con su política y, al final del episodio, actualiza sus redes a partir de las transiciones almacenadas.

Desde el punto de vista del software, la arquitectura separa cinco componentes con responsabilidades acotadas, lo que permite recombinarlos sin tocar el bucle de entrenamiento:

- **Mapa** (`GridMap`): representación discreta del entorno como una matriz binaria de ocupación con obstáculos rectangulares (`GridObstacle`).
- **Robot** (`Robot`, con dos realizaciones concretas): mantiene la posición, implementa el modelo de movimiento discreto y expone el conjunto de acciones válidas en cada estado.
- **Sensor** (`GridLidar`): *raycasting* 2D acelerado en PyTorch, con caché de distancias por mapa.
- **Entorno** (`BaseGridEnv` y subclases): envoltorio Gymnasium que coordina los tres anteriores, decide el fin del episodio y aplica la función de recompensa.
- **Agente** y **entrenador** (`BaseRLAgent`, `BaseTrainer`): el agente implementa la política y las actualizaciones del algoritmo concreto; el entrenador aloja el bucle de episodios y se comunica con el agente únicamente a través de ganchos (*hooks*) definidos en la clase base, lo que permite reutilizar un único bucle para DQN, PPO y el resto de algoritmos.

El punto clave del diseño es que la función de recompensa no está cableada en el entorno, sino que se le inyecta como un objeto que cumple el contrato `RewardFunction` (Sección 4.6). Así, el entorno decide *cuándo* termina un episodio (meta alcanzada, colisión o superación del horizonte temporal) y la recompensa sólo puntúa el paso, lo que permite comparar funciones de recompensa distintas sin alterar la semántica de la tasa de éxito.

4.4. Modelo cinemático del AGV sobre rejilla

El vehículo opera sobre una cuadrícula discreta de celda fija $\Delta_{\text{cell}} = 10$ unidades de simulación. La pose se expresa en coordenadas enteras $\mathbf{p}_t = (c_t, r_t) \in \mathbb{Z}^2$ (columna y fila; el origen está en la esquina superior izquierda del mapa y las filas crecen hacia abajo). Se definen dos modelos cinemáticos discretos alternativos.

4.4.1. Modelo con giro y avance (`GridRobot`)

El robot mantiene una orientación interna $\theta_t \in \{0°, 90°, 180°, 270°\}$ (N, O, S, E), y el estado se amplía a $\mathbf{x}_t = (c_t, r_t, \theta_t)$. La transición dado $a_t \in \{0, 1, 2\}$ es:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{f}(\mathbf{x}_t, a_t), \qquad \mathbf{f}(\mathbf{x}_t, a_t) = \begin{bmatrix} \delta(a_t)\,\mathbf{d}(\theta_t) \\ \Delta\theta(a_t) \end{bmatrix}, \tag{4.2}$$

con $\mathbf{d}(\theta_t) = (\cos\theta_t, -\sin\theta_t) \in \{-1, 0, 1\}^2$ el vector unitario de dirección cardinal (la componente de fila invierte el signo porque las filas crecen hacia abajo), $\delta(a_t) = \mathbb{I}(a_t = 0)$ el indicador de traslación, y

$$\Delta\theta(a_t) = \begin{cases} 0 & \text{si } a_t = 0 \text{ (avanzar)} \\ +90° \quad (\text{mód } 360°) & \text{si } a_t = 1 \text{ (giro izq.)} \\ -90° \quad (\text{mód } 360°) & \text{si } a_t = 2 \text{ (giro der.)} \end{cases} \tag{4.3}$$

Si la celda frontal está ocupada, el avance es un paso sin movimiento (se registra como colisión); los giros son siempre válidos. La figura 4.3 muestra el efecto de cada acción sobre el estado del robot en la rejilla de la figura 4.1.

4.4.2. Modelo sin giro con desplazamiento cardinal (`GridRobotNotTurning`)

Para este modelo la orientación es constante ($\theta = 0$) y el estado se reduce a la posición cartesiana $\mathbf{p}_t = (c_t, r_t) \in \mathbb{Z}^2$. La ecuación de movimiento adopta la forma matricial

$$\mathbf{p}_{t+1} = \mathbf{p}_t + \mathbf{M}\,\mathbf{e}_{a_t}, \tag{4.4}$$

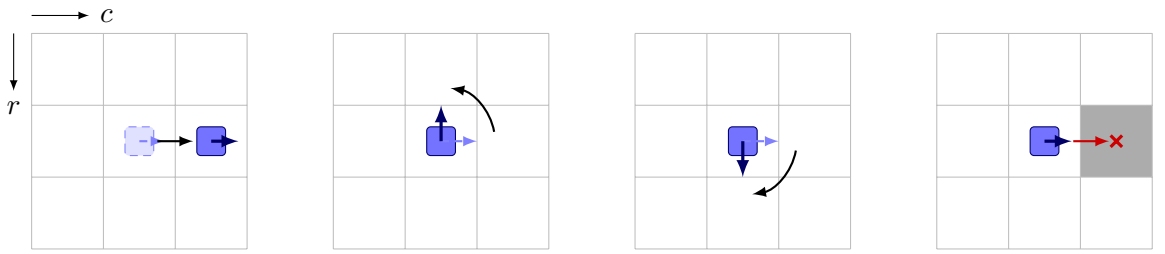


Figura 4.3: Acciones del modelo con giro (`GridRobot`) y estado resultante, partiendo en todos los casos del estado (c_t, r_t, θ_t) con el robot orientado hacia la derecha. El robot sombreado y la flecha discontinua indican el estado previo. Avanzar desplaza el robot a la celda frontal sin cambiar su orientación; los giros cambian la orientación sin desplazarlo. Si la celda frontal está ocupada, el avance se registra como colisión y el estado no cambia. Fuente: Realización propia.

donde $\mathbf{e}_{a_t} \in \{0,1\}^4$ es el vector canónico de la acción $a_t \in \{0,1,2,3\}$ y $\mathbf{M} \in \mathbb{Z}^{2\times 4}$ es la matriz de desplazamientos cardinales

$$\mathbf{M} = \begin{bmatrix} 0 & 0 & 1 & -1 \\ -1 & 1 & 0 & 0 \end{bmatrix} \qquad \text{columnas} = \{\mathrm{N}, \mathrm{S}, \mathrm{E}, \mathrm{O}\}. \tag{4.5}$$

Si la celda destino está fuera del mapa o es un obstáculo, el robot no se mueve y el entorno marca la transición como colisión. La figura 4.4 muestra el estado alcanzado con cada acción.

4.4.3. Elección del modelo cinemático

Los experimentos reportados en el capítulo de resultados utilizan el modelo sin giro (`GridRobotNotTurning`), mientras que el modelo con giro (`GridRobot`) se mantiene implementado como alternativa. La elección se justifica por tres motivos. En primer lugar, el espacio de estados cinemáticos del modelo sin giro es cuatro veces menor, ya que no incluye la orientación. En segundo lugar, cada acción válida produce un desplazamiento efectivo, de modo que la trayectoria entre dos celdas requiere, en general, menos pasos que en el modelo con giro, en el que los cambios de orientación consumen pasos adicionales sin acercar al robot a la meta. En tercer lugar, como consecuencia de lo anterior, el término de progreso r_{prog} (Sección 4.6) aporta información en todos los pasos sin colisión, mientras que en el modelo con giro se anula en cada rotación, lo que hace la señal de recompensa más dispersa y dificulta la asignación de crédito a lo largo del episodio.

Esta simplificación tiene un coste en realismo. El modelo sin giro corresponde a un vehículo holónomo, capaz de desplazarse en cualquier dirección sin reorientarse, como los AGV omnidireccionales con ruedas *mecanum*, pero no a los AGV de tracción diferencial, que deben girar antes de cambiar de dirección. Además, al ser la orientación constante, el sistema de

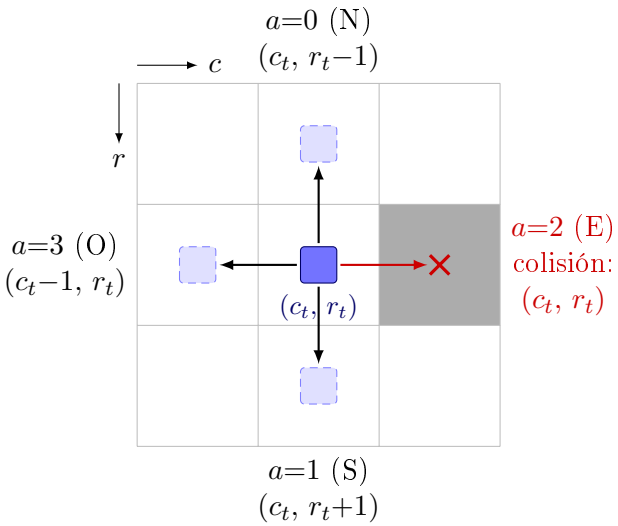


Figura 4.4: Acciones del modelo sin giro (`GridRobotNotTurning`) y estado resultante a partir de la posición (c_t, r_t). Cada acción desplaza el robot a una de las cuatro celdas vecinas (en discontinuo); como las filas crecen hacia abajo, moverse al norte reduce r_t. La acción que conduce a una celda ocupada (en rojo) provoca una colisión y el robot permanece en su posición. Fuente: Realización propia.

referencia en el que se expresan las lecturas del LiDAR y el rumbo a la meta (Sección 4.5) queda fijo respecto al mapa, por lo que la observación pierde su carácter egocéntrico. La evaluación sistemática del modelo con giro se plantea como línea de trabajo futuro.

4.5. Espacio de observaciones

En cada instante t, el agente recibe un vector denso $\mathbf{o}_t \in \mathbb{R}^{D_{\text{obs}}}$ formado por las lecturas del LiDAR agrupadas en sectores angulares, seguidas de la distancia y el rumbo relativos a la meta.

4.5.1. Sensor LiDAR por secciones angulares

El sensor `GridLidar` lanza N_{rays} rayos en un campo de visión FoV, con alcance máximo $d_{\text{máx}}$ y ruido aditivo gaussiano de desviación σ_{noise}. Para cada rayo devuelve la pareja (d_i, ϕ_i) con d_i la distancia al primer obstáculo (acotada a $d_{\text{máx}}$ si no hay impacto) y ϕ_i el ángulo del rayo relativo al rumbo del robot. Las distancias se cachean por mapa en tensores de PyTorch, lo que permite reusar el cálculo cuando el campo visual es 2π y el robot cambia de posición pero conserva orientación.

Para controlar la dimensión del estado observable sin perder cobertura angular, los N_{rays} rayos se agrupan en N_{sec} sectores angulares de amplitud uniforme. En cada sector se conserva

la lectura más próxima al robot:

$$(r^{(k)}, \phi^{(k)}) = \arg \min_{(d_i,\phi_i)\in \mathrm{Sector}_k} d_i, \qquad k = 1, \ldots, N_{\mathrm{sec}}. \tag{4.6}$$

Si un sector no detecta ningún obstáculo dentro de $d_{\mathrm{máx}}$, se asigna $(r^{(k)}, \phi^{(k)}) = (d_{\mathrm{máx}}, \bar{\phi}_k)$, con $\bar{\phi}_k$ el centro angular del sector. Las configuraciones típicas usadas en los experimentos son $N_{\mathrm{sec}} = 8$ con $N_{\mathrm{rays}} = 15\, N_{\mathrm{sec}}$, $d_{\mathrm{máx}} = 10$ celdas y FoV $= 2\pi$.

4.5.2. Variables relativas al objetivo

Dadas las posiciones del robot $\mathbf{p}_t$ y de la meta $\mathbf{p}_g$, el entorno calcula la distancia euclídea $d_{\mathrm{goal}} = \|\mathbf{p}_g - \mathbf{p}_t\|_2$ y el rumbo relativo al heading del robot

$$\theta_{\mathrm{goal}} = \mathrm{wrap}\Big(\mathrm{atan2}(-\Delta r,\, \Delta c) - \theta_t \Big) \in (-\pi, \pi], \tag{4.7}$$

con $(\Delta c, \Delta r) = \mathbf{p}_g - \mathbf{p}_t$ y $\mathrm{wrap}(\cdot)$ el envoltorio al intervalo $(-\pi, \pi]$. El signo negativo en Δr compensa que las filas crecen hacia abajo, de modo que la convención angular coincide con la del LiDAR (antihoraria, con el cero alineado con el heading del robot). Esta elección de convención unifica el frame egocéntrico de LiDAR y meta, y corrige un espejado que afectaba a los experimentos anteriores al test 25.

4.5.3. Vector de observación

Concatenando los sectores y las variables de la meta se obtiene la observación plana

$$\mathbf{o}_t = \Big[r^{(1)}, \phi^{(1)}, \ldots, r^{(N_{\mathrm{sec}})}, \phi^{(N_{\mathrm{sec}})}, d_{\mathrm{goal}},\, \theta_{\mathrm{goal}} \Big]^{\top} \in \mathbb{R}^{2N_{\mathrm{sec}}+2}. \tag{4.8}$$

Para $N_{\mathrm{sec}} = 8$, la dimensión del estado observable es $D_{\mathrm{obs}} = 18$. El entorno expone adicionalmente, en la información auxiliar de cada paso, la máscara binaria de las acciones válidas $m_t \in \{0, 1\}^{|\mathcal{A}|}$ obtenida por el robot (`Robot.valid_actions`), utilizada por el mecanismo de enmascarado descrito en la Sección 4.9.

4.6. Función de recompensa

La señal de recompensa se construye como una combinación lineal de cuatro términos con pesos parametrizables [Gao et al., 2025, Xue et al., 2022]:

$$r_t = r_{\mathrm{prog}} + r_{\mathrm{step}} + r_{\mathrm{prox}} + r_{\mathrm{term}}. \tag{4.9}$$

- **Progreso hacia la meta.** Mide la aproximación radial normalizada a la meta respecto

a la iteración anterior:

$$r_{\text{prog}} = w_d \, \frac{d_{t-1} - d_t}{d_{\text{diag}}}, \tag{4.10}$$

con $d_t = d_{\text{goal}}$, d_{diag} la diagonal máxima sobre todos los mapas de entrenamiento (invariable, para que las escalas sean comparables entre mapas) y w_d un peso configurable. En los experimentos de referencia se emplea $w_d = 10$ y la variante *clipped* $r_{\text{prog}} \leftarrow \max(r_{\text{prog}}, 0)$, que anula la señal cuando el robot se aleja de la meta en lugar de penalizarla; una variante alternativa reemplaza d_t por la distancia geodésica calculada por un BFS 4-conexo desde la celda meta.

- **Coste por paso.** Penalización escalar constante que desincentiva oscilaciones y rutas excesivamente largas:

$$r_{\text{step}} = -c_{\text{step}}, \tag{4.11}$$

con $c_{\text{step}} = 0{,}05$ en los experimentos de referencia.

- **Penalización por proximidad.** Activa cuando la lectura más cercana del LiDAR cae por debajo de un margen de seguridad D_{safe}; crece linealmente desde cero hasta $-\lambda_{\text{prox}}$ al tocar el obstáculo:

$$r_{\text{prox}} = \begin{cases} -\lambda_{\text{prox}} \, \dfrac{D_{\text{safe}} - d_{\text{mín}}}{D_{\text{safe}}} & \text{si } d_{\text{mín}} < D_{\text{safe}} \\ 0 & \text{en otro caso} \end{cases}, \tag{4.12}$$

con $d_{\text{mín}} = \min_k r^{(k)}$. En los experimentos de referencia se utiliza $\lambda_{\text{prox}} = 10^{-2}$ y $D_{\text{safe}} = \frac{2}{3} \, d_{\text{máx}}$.

- **Recompensa terminal.** Se suma al final del paso cuando la transición cierra el episodio:

$$r_{\text{term}} = \begin{cases} +R_{\text{win}} & \text{si } \|\mathbf{p}_g - \mathbf{p}_t\|_2 < \varepsilon_{\text{goal}} \\ -R_{\text{fail}} & \text{si el robot colisiona} \\ 0 & \text{en una transición regular o por fin de horizonte} \end{cases}, \tag{4.13}$$

con $R_{\text{win}} = 20$, $R_{\text{fail}} = 5$ y $\varepsilon_{\text{goal}} = 1$ celda. El agotamiento del horizonte temporal $T_{\text{máx}}$ no concede recompensa terminal, por lo que la señal de fin por tiempo se transmite al agente únicamente a través del acumulado de los c_{step}.

Esta formulación corresponde con `RewardFunction`, descrito en la Sección 4.3: el entorno construye un `StepContext` con toda la información del paso (d_t, d_{t-1}, $d_{\text{mín}}$, $\mathbf{p}_t$, $\mathbf{p}_g$, acción previa, flags de meta/colisión, etc.) y la función de recompensa sólo lo puntúa. En consecuencia, la tasa de éxito es independiente del diseño de la recompensa y permite comparar funciones alternativas sobre la misma métrica.

4.7. Arquitecturas de red neuronal

Las políticas y las funciones de valor se aproximan con perceptrones multicapa (MLP) completamente conectados definidos en el módulo `networks.py`. La dimensión de la capa de entrada, $D_{\text{obs}} = 2N_{\text{sec}} + 2$, varía con la resolución angular del LiDAR; la de salida se ajusta a la cardinalidad del espacio de acciones ($|\mathcal{A}| \in \{3, 4\}$). La configuración empleada en los experimentos de referencia se resume en el Cuadro 4.1.

Tabla 4.1: Arquitecturas de red neuronal implementadas y utilizadas por los agentes DQN y PPO. La red llamada `SimpleQNetwork` se emplea tanto como aproximador $Q(s, a)$ en DQN como para producir los logits de la política en PPO; la red `ValueNetwork` actúa como crítico $V(s)$ en PPO. Las dimensiones aquí reflejan el baseline experimental; todas las redes son paramétricas en profundidad y anchura.

Red	**Capa / Tipo**	**Dim.**	**Activación**
`GridAgentNet`	Entrada	D_{obs}	—
(DQN base)	Oculta 1 (Lineal)	253	ReLU
	Oculta 2 (Lineal)	56	ReLU
	Salida (Lineal)	$\lvert\mathcal{A}\rvert$	— ($Q(s, a)$)
`SimpleQNetwork`	Entrada	D_{obs}	—
(DQN / política PPO)	Oculta $1 \dots L_{\text{hid}} + 1$	$H = 64$	ReLU
	Salida (Lineal)	$\lvert\mathcal{A}\rvert$	— (logits o Q)
`ValueNetwork`	Entrada	D_{obs}	—
(crítico PPO)	Oculta 1 (Lineal)	$H = 64$	ReLU
	Oculta 2 (Lineal)	64	ReLU
	Salida (Lineal)	1	— ($V(s)$)

4.7.1. Aproximador de valor de acción (DQN)

La familia de agentes basada en valor (clases `GridAgent` y `DQNAgent`) aproxima la función de valor óptima $Q^*(s, a)$ mediante una red $Q_\theta(s, a)$ y utiliza Double DQN para desacoplar la selección de la acción de su evaluación [Shakya et al., 2023]. Durante la inferencia, la acción se obtiene de manera determinista como $a_t = \arg\max_a Q_\theta(\mathbf{o}_t, a)$. Durante el entrenamiento, la exploración se controla mediante una política ε-greedy con decaimiento exponencial

$$\varepsilon_t = \varepsilon_{\text{mín}} + (\varepsilon_{\text{máx}} - \varepsilon_{\text{mín}}) \exp\left(-\frac{t}{\tau_\varepsilon}\right), \tag{4.14}$$

con valores por defecto $\varepsilon_{\text{máx}} = 1$, $\varepsilon_{\text{mín}} = 0{,}05$ y $\tau_\varepsilon = 10^5$ pasos de simulación. Las transiciones se almacenan en un *buffer* de repetición de experiencia de capacidad 10^5 y la red objetivo se sincroniza cada $N_{\text{tgt}} = 10^3$ actualizaciones. La red soporta tanto la variante base `GridAgentNet` (dos capas ocultas asimétricas, $253 \to 56$) como la variante paramétrica

`SimpleQNetwork` con profundidad y anchura configurables.

4.7.2. Actor-crítico con PPO

El agente `PPOAgent` implementa optimización de política próxima con el objetivo recortado

$$\mathcal{L}^{\mathrm{CLIP}}(\theta) = \hat{\mathbb{E}}_t\Big[\min\big(\rho_t(\theta)\,\hat{A}_t,\ \mathrm{clip}(\rho_t(\theta),\,1-\varepsilon,\,1+\varepsilon)\,\hat{A}_t\big)\Big], \tag{4.15}$$

con el cociente de probabilidades $\rho_t(\theta) = \pi_\theta(a_t \mid \mathbf{o}_t)/\pi_{\theta_{\mathrm{old}}}(a_t \mid \mathbf{o}_t)$ y las ventajas $\hat{A}_t$ estimadas por GAE [Sivashangaran and Eskandarian, 2023, Ge et al., 2024]. La pérdida total añade un término de valor y un término de entropía para fomentar la exploración:

$$\mathcal{L}(\theta,\phi) = \mathcal{L}^{\mathrm{CLIP}}(\theta) + c_v\,\big(V_\phi(\mathbf{o}_t) - R_t\big)^2 - c_H\,\mathcal{H}\big[\pi_\theta(\cdot \mid \mathbf{o}_t)\big]. \tag{4.16}$$

La implementación concreta tiene dos particularidades que conviene documentar y que diferencian este PPO del esquema estrictamente *on-policy* descrito en la literatura introductoria:

1. **Normalización del crítico**: la red `ValueNetwork` devuelve tensores de forma $(B, 1)$, mientras que los retornos tienen forma $(B,)$. Las versiones anteriores al experimento de referencia comparaban cada retorno con cada valor (producto exterior), lo que degradaba el aprendizaje del crítico. En la implementación actual se aplica una reducción explícita $V_\phi(\mathbf{o}_t)$.squeeze(-1) antes del cálculo de la pérdida.

2. **Reuso de transiciones**: aunque las ventajas y los logaritmos de probabilidad se calculan con las muestras del episodio recién terminado mediante GAE, cada transición se añade a un *buffer* circular de capacidad 10^5 del que se muestrea un mini-lote de 64 transiciones por actualización. En consecuencia, la actualización utiliza transiciones de episodios anteriores con log-probabilidades y ventajas estimadas sobre políticas antiguas, lo que acerca el comportamiento del método al de un algoritmo *off-policy*. Esta decisión se mantiene porque reproduce la configuración empleada en los experimentos de referencia; en el análisis de resultados se discute su efecto sobre la estabilidad y la eficiencia.

El actor (`SimpleQNetwork` usada como red de política) produce los logits de una distribución categórica sobre las acciones, de la que se muestrea a_t durante el entrenamiento y de la que se toma la moda (arg máx) durante la evaluación *greedy*. El crítico (`ValueNetwork`) estima $V_\phi(\mathbf{o}_t)$ y se entrena para minimizar el error cuadrático medio frente a los retornos descontados observados.

Los hiperparámetros empleados en los experimentos de referencia se detallan en el capítulo de resultados.

4.8. Aprendizaje por currículo (opcional)

Para mitigar la dispersión de la recompensa en las primeras etapas del entrenamiento se puede activar un currículo automático (`SuccessCurriculum`) inspirado en *NavACL* [Xue et al., 2022]. El currículo se compone de una secuencia de etapas $\mathcal{E}_1, \ldots, \mathcal{E}_K$, cada una definida por el conjunto de mapas permitidos, una distancia máxima inicial robot-meta, un umbral de tasa de éxito, un número mínimo de episodios por etapa y un número máximo opcional. En cada episodio el entorno pregunta al currículo qué mapa puede seleccionar y qué distancia inicial máxima es admisible al muestrear la meta; al final del episodio, informa al currículo del resultado (éxito o fallo). Cuando la tasa de éxito en los últimos W episodios (ventana por defecto $W = 500$) alcanza el umbral de la etapa y se ha superado el número mínimo de episodios, el currículo avanza a la siguiente. Si una etapa no se supera en $N_{\text{máx}}$ episodios, se avanza por tiempo. La última etapa se mantiene hasta el final del entrenamiento.

El currículo no modifica ni al agente ni al bucle de entrenamiento: su estado se consulta exclusivamente desde el entorno (`BaseGridEnv(curriculum=...)`), y el entrenador se limita a registrar la etapa actual en la salida de *TensorBoard*.

4.9. Enmascarado de acciones inválidas (opcional)

Cada robot sabe calcular, para un estado dado, qué acciones conducen a una colisión. Esta información se expone como una máscara binaria $m_t \in \{0, 1\}^{|\mathcal{A}|}$ en la información auxiliar del entorno. Cuando se activa el enmascarado (`TrainerConfig(action_masking=True)`), el entrenador pasa la máscara a `PPOAgent.act(..., action_mask=...)`: los *logits* de la política se corrigen con -10^9 en las acciones inválidas antes de muestrear, y la misma corrección se aplica durante el cálculo de la pérdida PPO para que las log-probabilidades sean consistentes entre el muestreo y la actualización. El enmascarado elimina las colisiones por diseño, pero no las trampas que requieren alejarse momentáneamente de la meta para rodear un obstáculo (mínimos locales), por lo que no sustituye al diseño de la recompensa.

4.10. Extensión recurrente (opcional)

Si la observación local no es suficiente para desambiguar estados geométricamente idénticos, la red de política puede ampliarse con una capa recurrente que procese la historia de observaciones y acciones. En el repositorio se han implementado dos variantes: `PPOGRUAgent`, que mantiene un estado oculto a lo largo del episodio pero actualiza con transiciones individuales, y `RecurrentPPOAgent`, que trocea cada episodio en secuencias de longitud fija (*sequence-chunked GAE*) y actualiza la red con el gradiente propagado a través de la secuencia. Ambas emplean una única capa GRU para evitar el desvanecimiento del gradiente observado en redes recurrentes más profundas sobre entornos discretos. El análisis experi-

mental de estas variantes queda fuera del alcance de la comparación principal DQN/PPO y se presenta como estudio complementario en el capítulo de resultados.

Capítulo 5

Configuración experimental y resultados

En este capítulo se presentan la configuración experimental y los resultados obtenidos al entrenar y evaluar los agentes descritos en el Capítulo 4. Primero se detallan los escenarios de la rejilla, los hiperparámetros de los algoritmos, el entorno de ejecución y el protocolo de evaluación. A continuación se presentan los resultados del agente PPO de referencia (test 25), seguidos de una serie de estudios complementarios que modifican un único elemento del baseline para medir su efecto: aprendizaje por currículo, reward geodésico, política recurrente (GRU), alcance del LiDAR y enmascarado de acciones inválidas. Al final del capítulo se discuten los patrones observados.

5.1. Configuración experimental

5.1.1. Escenarios de la rejilla

Los experimentos se realizan sobre cuatro mapas estáticos de ocupación, almacenados como ficheros JSON y cargados en tiempo de ejecución. Durante el entrenamiento, el mapa se selecciona de manera uniforme al inicio de cada episodio (variante multi-mapa del entorno `Grid_Robot_Sections_Env_MultiMap`); durante la evaluación, el protocolo se describe en la Sección 5.1.7. Los mapas empleados se resumen en el Cuadro 5.1.

Tabla 5.1: Mapas de ocupación empleados en los experimentos. Las celdas ocupadas incluyen tanto los obstáculos declarados como su posible solapamiento.

Mapa	**Descripción**	**Celdas ocupa**
`map_obstacle_free`	Rejilla vacía, caso de control.	0
`map_1_obstacle`	Un obstáculo asimétrico.	~ 20
`map_4_square_obstacle`	Cuatro obstáculos rectangulares.	~ 60
`map_simple`	14 obstáculos solapados que forman un bolsillo cóncavo.	121

El escenario `map_obstacle_free` valida que la política aprenda el término de progreso aislado del resto de términos de la recompensa. Los escenarios con obstáculos aumentan

progresivamente la densidad geométrica; `map_simple` es el más exigente porque contiene una geometría cóncava en la que la política reactiva debe alejarse momentáneamente de la meta para rodear el bolsillo, lo que configura precisamente el modo de fallo por mínimo local descrito por Gao et al. [2025].

5.1.2. Hiperparámetros del agente PPO

El experimento de referencia (test 25) entrena el agente `PPOAgent` con los hiperparámetros del Cuadro 5.2. Todos los experimentos posteriores (tests 26–31) heredan esta misma configuración y modifican un único elemento, descrito en cada apartado.

Tabla 5.2: Hiperparámetros del agente PPO en el experimento de referencia (test 25).

Hiperparámetro	**Valor**	**Descripción**
Tasa de aprendizaje α_{lr}	10^{-3}	Optimizador Adam, igual para actor y crítico.
Factor de descuento γ	0,99	Descuento temporal del retorno.
λ de GAE	0,95	Suavizado del estimador de ventajas.
Clip PPO ε	0,20	Recorte del objetivo *surrogate*.
Coef. de valor c_v	0,50	Peso de la pérdida del crítico.
Coef. de entropía c_H	0,01	Peso del término de entropía.
Tamaño de lote	64	Mini-lote del optimizador.
Capacidad del *buffer*	10^5	*Buffer* circular de transiciones.
Norma máx. de gradiente	5,0	*Gradient clipping* aplicado al optimizador.
Episodios de entrenamiento	40 000	Horizonte total.
Horizonte por episodio	400 pasos	Trunca el episodio si no se alcanza la meta antes.
Pasos por actualización	4	`TrainerConfig.eps_step`, actualiza cada 4 pasos.

5.1.3. Hiperparámetros del agente DQN

Para el agente DQN basado en `GridAgent` se emplean los hiperparámetros del Cuadro 5.3, utilizando la variante Double DQN con red objetivo congelada.

5.1.4. Función de recompensa

Todos los experimentos emplean la función `ProgressReward` descrita en la Sección 4.6, con los parámetros del Cuadro 5.4 salvo donde se indique lo contrario (test 27 activa la variante geodésica).

5.1.5. Configuración del sensor LiDAR

El sensor `GridLidar` se configura con alcance máximo $d_{\text{máx}} = 10$ celdas, campo de visión 2π y ruido gaussiano aditivo $\sigma_{\text{noise}} = 10^{-3}$ celdas. El número total de rayos es $N_{\text{rays}} = 15\,N_{\text{sec}}$

Tabla 5.3: Hiperparámetros del agente DQN (`GridAgent`).

Hiperparámetro	**Valor**	**Descripción**
Tasa de aprendizaje α_{lr}	10^{-3}	Optimizador Adam.
Factor de descuento γ	0,99	Descuento temporal del retorno.
$\varepsilon_{\text{máx}}$	1,00	Probabilidad inicial de exploración.
$\varepsilon_{\text{mín}}$	0,05	Probabilidad asintótica de exploración.
τ_ε	10^5 pasos	Constante de decaimiento exponencial de ε.
Frec. actualización red objetivo	10^3 pasos	Sincroniza $\theta^- \leftarrow \theta$.
Tamaño de lote	64	Mini-lote del optimizador.
Capacidad del *buffer*	10^5	*Buffer* circular de transiciones.
Norma máx. de gradiente	5,0	*Gradient clipping*.
Variante	Double DQN	Desacopla selección y evaluación de acciones.

Tabla 5.4: Parámetros de la función de recompensa `ProgressReward` en el experimento de referencia.

Parámetro	**Valor**
Escala de progreso w_d	10
Coste por paso c_{step}	0,05
Margen de seguridad D_{safe}	$\frac{2}{3}\, d_{\text{máx}}$ ($\approx 6{,}67$ celdas)
Penalización máx. por proximidad λ_{prox}	0,01
Recompensa terminal por meta R_{win}	$+20$
Penalización por colisión R_{fail}	-5
Tolerancia a la meta $\varepsilon_{\text{goal}}$	1 celda
Progreso negativo recortado a cero	Sí
Métrica de distancia	Euclídea (geodésica en test 27)

para garantizar varias muestras por sector; en el experimento de referencia $N_{\text{sec}} = 8$ (y por tanto $N_{\text{rays}} = 120$). El sensor mantiene una caché de distancias por mapa, acelerada en GPU, que se reutiliza entre episodios siempre que el campo de visión sea 2π.

5.1.6. Entorno de ejecución

El entrenamiento se ejecuta en el equipo del autor con una tarjeta NVIDIA compatible con CUDA; cuando no hay GPU disponible, el agente y el sensor delegan en la CPU (en MPS si se detecta un Apple Silicon). El tiempo total del test 25 fue de 5 h 49 min para 40 000 episodios en GPU, lo que da una velocidad media aproximada de 115 episodios por minuto. La capa de simulación emplea `gymnasium` 0.29+ y PyTorch; el *logging* de métricas se realiza con `TensorBoard` y la configuración de cada ejecución se serializa en un fichero `config.json` dentro del directorio `runs/Test/<N>/`.

5.1.7. Protocolo de evaluación

Al final del entrenamiento, el agente se evalúa durante 1 000 episodios sobre los cuatro mapas, con la semilla 0 y en dos modalidades complementarias:

- **Modo *sampled***: la acción se muestrea de la distribución categórica del actor, replicando el comportamiento estocástico del entrenamiento.
- **Modo *greedy***: la acción se escoge deterministamente como el arg máx de los *logits* del actor; refleja el comportamiento más conservador que cabe esperar en despliegue.

Se reportan las siguientes métricas globales y por mapa:

- Tasa de éxito (*Success rate*, SR): porcentaje de episodios terminados por alcanzar la meta.
- Tasa de colisión y tasa de agotamiento del horizonte temporal.
- *Loops*: número de episodios en los que la trayectoria visita alguna celda al menos veinte veces, indicador de ciclos estables (típicos de la modalidad *greedy*).
- *Path ratio*: cociente entre los pasos efectivos del episodio y el camino óptimo calculado por BFS; valores próximos a uno indican trayectorias eficientes.

Las comparaciones entre experimentos emplean el test z de proporciones de dos muestras ($|z| > 1{,}96$ equivale a significación estadística al 5 %).

5.2. Resultados del baseline PPO (test 25)

El agente PPO del baseline alcanza una tasa de éxito acumulada del 83 % en los primeros 4 000 episodios y asciende hasta el 93 % al llegar a los 40 000 episodios, con la entropía de la política todavía en 0,55 nats al final del entrenamiento. En la evaluación posterior (1 000 episodios, semilla 0), los resultados globales se recogen en el Cuadro 5.5.

Tabla 5.5: Resultados del agente PPO de referencia (test 25) en la evaluación final sobre los cuatro mapas (1 000 episodios por modalidad, semilla 0). Los *loops* se cuentan sobre los episodios que agotan el horizonte temporal.

Modalidad	**SR global (%)**	**Colisiones (%)**	**Timeouts (%)**	**Loops**
Sampled	93,9	5,1	1,0	3
Greedy	91,1	1,1	7,8	76

En la evaluación por mapa (Cuadro 5.6), tres de los cuatro escenarios superan el 91 % de éxito en ambas modalidades, mientras que `map_simple` se estabiliza en torno al 75–78 % según la modalidad. La diferencia entre modalidades es especialmente visible en `map_4_square_obstacle`, que pasa de 91,4 % en *greedy* a 99,2 % al muestrear la política. Dicho de otro modo: el modo *sampled* cambia los *timeouts* del *greedy* por un número mayor de colisiones, lo que se traduce en una tasa de éxito global más alta (93,9 % frente a 91,1 %) pero a costa de más pasos cerca de los obstáculos.

El análisis de las trayectorias fallidas muestra que la mayor parte de las colisiones en `map_simple` se producen con el LiDAR detectando el obstáculo a una sola celda de distancia en la dirección de avance: no es un problema de percepción, sino de ruido residual de la política junto a una pared, que en la modalidad *greedy* degenera en ciclos estables (76 de los 78 *timeouts* son *loops*). Las trayectorias que sí alcanzan la meta son prácticamente óptimas: la mediana del cociente *pasos / BFS* vale 1,0 en *greedy* para los cuatro mapas y oscila entre 1,0 y 1,1 en *sampled*.

Tabla 5.6: Tasa de éxito por mapa del baseline PPO (test 25). Entre paréntesis, el número de episodios con el mapa seleccionado sobre el total de 1 000.

Mapa	**Episodios**	**SR *sampled* (%)**	**SR *greedy* (%)**
`map_obstacle_free`	267	99,6	99,6
`map_1_obstacle`	230	98,7	98,7
`map_4_square_obstacle`	255	99,2	91,4
`map_simple`	248	77,8	74,6
Global	1 000	**93,9**	**91,1**

5.3. Estudios complementarios

Los experimentos siguientes modifican un único elemento del baseline para aislar su efecto. El Cuadro 5.7 resume los resultados globales; los apartados posteriores detallan cada estudio.

Tabla 5.7: Comparación de los experimentos complementarios frente al baseline PPO (test 25). Las columnas z reportan el estadístico del test de proporciones frente al baseline sobre 1 000 episodios por modalidad; $|z| > 1{,}96$ equivale a significación al 5 %.

Test	Variante	SR *sampled* (%)	SR *greedy* (%)	SR map_simple (%)	Tier
25	Baseline PPO	93,9	91,1	77,8	5 h 49
26	+ currículo (5 etapas)	93,9 ($z=0$)	89,3 (n.s.)	77,8 (n.s.)	3 h 18
27	reward geodésico	93,4 (n.s.)	85,0 ($z=-4{,}2$)	78,6 (n.s.)	2 h 52
28	GRU, sin *grad clip*	85,2	77,4	~ 60	4 h 37
29	GRU, con *grad clip*	89,8 ($z=-3{,}4$)	88,3 ($z=-2{,}1$)	~ 70	2 h 51
30	LiDAR $d_{\text{máx}} = 30$	93,7 (n.s.)	90,4 (n.s.)	79,4 (n.s.)	2 h 41
31	+ *action masking*	*pendiente de entrenamiento*			–

5.3.1. Aprendizaje por currículo (test 26)

El test 26 añade al baseline un currículo `SuccessCurriculum` de cinco etapas que empieza en el mapa vacío y añade progresivamente los mapas con obstáculos, con un umbral de éxito del 90 % (etapas 1–4) y del 85 % (etapa final) medido sobre una ventana de 500 episodios. Las etapas se activan en los episodios 0/1620/2120/3416/3916, de modo que el agente llega al escenario completo tras unos 4 000 episodios. El currículo reduce sensiblemente el tiempo necesario para alcanzar el 80 % de éxito, pero la política final es estadísticamente indistinguible del baseline (SR *sampled* 93,9 % en ambos). El resultado concuerda con los informes de Xue et al. [2022], en los que la ganancia del currículo se concentra en la fase inicial del entrenamiento.

5.3.2. Progreso geodésico (test 27)

El test 27 sustituye la distancia euclídea del término de progreso por la distancia geodésica (BFS 4-conexo desde la celda meta, respetando los obstáculos). La hipótesis era que señalar al agente el camino que rodea el obstáculo mejoraría el rendimiento en `map_simple`. En la práctica, el modo *sampled* no mejora de forma significativa (SR 93,4 % frente a 93,9 %) y el modo *greedy* empeora de manera clara (SR 85,0 % frente a 91,1 %, $z = -4{,}2$) con un aumento notable de los *loops* (137 frente a 76). La señal geodésica parece inducir políticas determinísticas que explotan trayectorias de bordes y quedan atrapadas en ciclos cuando se evalúan en modo *greedy*.

5.3.3. Política recurrente con GRU (tests 28 y 29)

Los tests 28 y 29 evalúan la variante `RecurrentPPOAgent` descrita en la Sección 4.10, con una capa GRU de 40 unidades y secuencias de longitud 32. El test 28 reutilizó el bucle de entrenamiento del PPO estándar y descubrió que la clase recurrente no aplicaba el recorte de gradiente ni la máscara de padding; esto produjo un colapso del rendimiento desde el 90 % hasta el 25 % alrededor del episodio 25 000. El test 29 corrige ambos problemas y reduce la tasa de aprendizaje a $3{\cdot}10^{-4}$. El entrenamiento es entonces estable, pero la política final queda por debajo del baseline MLP (SR *sampled* 89,8 %, $z = -3{,}4$) con una entropía mucho menor (0,27 nats) y entre 95 y 111 *loops* en ambas modalidades. Las trayectorias exitosas son casi óptimas (mediana del *path ratio* = 1,00), pero la memoria recurrente, tal como está implementada, no elimina los ciclos. Este estudio no permite concluir que la memoria no aporte valor en general; sí indica que, con esta implementación y presupuesto de entrenamiento, el MLP sigue siendo competitivo.

5.3.4. Alcance extendido del LiDAR (test 30)

El test 30 aumenta el alcance del sensor a $d_{\text{máx}} = 30$ celdas (frente a 10 en el baseline), manteniendo el resto de parámetros constantes. El margen de seguridad relativo al nuevo alcance se recalcula para que siga valiendo 6,67 celdas en términos absolutos. El resultado es estadísticamente indistinguible del baseline en todas las métricas (SR *sampled* 93,7 %, $z = -0{,}2$), lo que sugiere que la limitación principal del baseline no reside en la percepción.

5.3.5. Enmascarado de acciones inválidas (test 31)

El test 31 activa el enmascarado descrito en la Sección 4.9 manteniendo el resto de la configuración del baseline. En una prueba de humo de 40 episodios no se registraron colisiones. El experimento está preparado para entrenamiento completo; los resultados definitivos se incorporarán a este capítulo cuando esté disponible la ejecución de 40 000 episodios. Se espera que el enmascarado reduzca a cero las colisiones y no modifique de manera sustancial el comportamiento en los episodios de mínimo local, en los que ninguna acción individual lleva al choque y, por tanto, la máscara no restringe el espacio efectivo de decisión.

5.4. Discusión

Los resultados experimentales permiten extraer varias observaciones sobre los límites del enfoque adoptado y sobre la relación entre las decisiones de diseño del Capítulo 4 y el comportamiento observado.

Casi todas las variantes convergen a un rendimiento similar. Los tests 25 (baseline), 26 (currículo), 27 (reward geodésico) y 30 (LiDAR extendido) terminan en una ventana

estrecha de tasa de éxito global ($\sim 93\,\%$ en modo *sampled*). Las variantes recurrentes (28 y 29) son algo peores. La convergencia a un rendimiento similar sugiere que, con el presupuesto de interacción estudiado, el cuello de botella no se encuentra en el aproximador ni en la percepción, sino en la señal de aprendizaje y en los patrones geométricos a los que la política no sabe responder.

El modo de fallo persistente es un mínimo local geométrico. `map_simple` se estabiliza en torno al 78 % en todas las variantes probadas. Las trayectorias fallidas siguen el patrón descrito por Gao et al. [2025]: el robot queda detenido o entra en un ciclo frente a un bolsillo cóncavo porque la salida requiere alejarse transitoriamente de la meta, acción que la señal densa de progreso desincentiva. Ni el currículo (que ajusta la distribución de tareas pero no modifica la función de recompensa) ni la señal geodésica (que dirige la política hacia la salida pero genera ciclos deterministas en *greedy*) resuelven este modo de fallo, y la memoria GRU implementada en los tests 28–29 tampoco lo hace.

La modalidad de evaluación cambia la lectura. El modo *greedy* elimina la variabilidad útil que, en modo *sampled*, saca al agente de los ciclos: el baseline mejora de 91,1 % a 93,9 % al pasar de *greedy* a *sampled*, y el test 27 muestra la mayor brecha (85,0 % frente a 93,4 %). Esta diferencia es relevante para una futura evaluación en un sistema real, donde una política demasiado determinista puede quedar atrapada en *loops* que no aparecerían con un muestreo estocástico.

Compromiso entre reutilización de muestras y estabilidad. La implementación de PPO empleada (Sección 4.7) muestrea mini-lotes de un *buffer* con transiciones de episodios anteriores, acercándose a un algoritmo *off-policy*. Esta decisión permite al agente aprovechar transiciones recolectadas con versiones ligeramente antiguas de la política, con el coste de que los log-probabilidades utilizados en la actualización pueden quedar obsoletos. En el baseline el efecto no bloquea el aprendizaje, pero en las variantes recurrentes se combina con la ausencia inicial de recorte de gradiente para producir el colapso observado en el test 28. Una línea de trabajo futuro inmediata consiste en comparar esta implementación con un PPO estrictamente *on-policy* sobre el mismo escenario, para medir la contribución específica del *buffer*.

Validez interna y limitaciones. Las conclusiones de este capítulo se circunscriben al robot `GridRobotNotTurning`, a los cuatro mapas descritos, al algoritmo PPO con los hiperparámetros documentados y a los 40 000 episodios de entrenamiento. Las cifras reportadas corresponden a una única semilla (0); una estimación de la varianza entre semillas, así como la comparación sistemática con el agente DQN de la Sección 4.7, quedan como líneas de trabajo pendientes antes de la defensa.

Capítulo 6

Conclusiones y trabajos futuros

Bibliografía

Hao Hu, Xiaoliang Jia, Qixuan He, Shifeng Fu, and Kuo Liu. Deep reinforcement learning based agvs real-time scheduling with mixed rule for flexible shop floor in industry 4.0. *Computers & Industrial Engineering*, 149:106749, 2020. doi: 10.1016/j.cie.2020.106749.

Xianfeng Ye, Zhiyun Deng, Yanjun Shi, and Weiming Shen. Toward energy-efficient routing of multiple agvs with multi-agent reinforcement learning. *Sensors*, 23(12):5615, 2023. doi: 10.3390/s23125615.

Shathushan Sivashangaran and Azim Eskandarian. Deep reinforcement learning for autonomous ground vehicle exploration without a-priori maps. *Advances in Artificial Intelligence and Machine Learning*, 3(2):1198–1219, 2023. arXiv preprint arXiv:2301.04036.

David Abad Pérez, Basil Mohammed Al-Hadithi, and Victor Cadix Martín. Navigation of mobile robots using neural networks and genetic algorithms. *IEEE Latin American Transactions*, 2022. URL `https://latamt.ieeer9.org/index.php/transactions/article/view/8506`.

Ashish Kumar Shakya, Gopinatha Pillai, and Sohom Chakrabarty. Reinforcement learning algorithms: A brief survey. *Expert Systems with Applications*, 231:120495, 2023. doi: 10.1016/j.eswa.2023.120495.

Kevin P. Murphy. *Reinforcement Learning: An Overview*. MIT Press, 2025.

Honghu Xue, Benedikt Hein, Mohamed Bakr, Georg Schildbach, Bengt Abel, and Elmar Rueckert. Using deep reinforcement learning with automatic curriculum learning for mapless navigation in intralogistics. *Applied Sciences*, 12(6):3153, 2022. doi: 10.3390/app12063153.

Dayu Guo, Yuan Zhu, and Ke Lu. Map-less navigation algorithm for autonomous vehicles based on deep reinforcement learning. *Advances in Computer, Signals and Systems*, 9(1): 123–130, 2025. doi: 10.23977/acss.2025.090117.

Jianqi Gao, Xizheng Pang, Qi Liu, and Yanjie Li. Hierarchical reinforcement learning for safe mapless navigation with congestion estimation. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025.

Abdul Quadir Md, Dibyanshu Jaiswal, Senthilkumar Mohan, Nisreen Innab, Riza Sulaiman, Mohammed Kbiri Alaoui, and Ali Ahmadian. A novel approach for self-driving car in partially observable environment using life long reinforcement learning. *Sustainable Energy, Grids and Networks*, 38:101356, 2024. doi: 10.1016/j.segan.2024.101356.

Lun Ge, Xiaoguang Zhou, Yongqiang Li, and Yongcong Wang. Deep reinforcement learning navigation via decision transformer in autonomous driving. *Frontiers in Neurorobotics*, 18:1338189, 2024. doi: 10.3389/fnbot.2024.1338189.

Martín Augusto Reigadas Terán. Modelos cinemáticos discretos del agv (`Robot`, `GridRobot`, `GridRobotNotTurning`). Repositorio de código del TFM, `src/tfm_control_ucm/core/grid_env/robot.py`, 2026a.

Martín Augusto Reigadas Terán. Script de entrenamiento `grid_training_19.py` (ppo, 8 secciones, map_1_obstacle). Repositorio de código del TFM, `train/grid_training_19.py`, 2026b.

Ibomoiye Domor Mienye, Theo G. Swart, and George Obaido. Recurrent neural networks: A comprehensive review of architectures, variants, and applications. *Information*, 15(9): 517, 2024. doi: 10.3390/info15090517.

Danilo Dessi, Francesco Osborne, Diego Reforgiato Recupero, Davide Buscaldi, Enrico Motta, and Harld Sack. Ai-kg: an automatically generated knowledge graph of artificial intelligence, 2020.

P. Chen, Y. Lu, V. W. Zheng, X. Chen, and B. Yang. Knowedu: A system to construct knowledge graph for education. *IEEE Access*, 6:31553–31563, 2018. doi: 10.1109/ACCESS.2018.2839607.

Yuehua Qin, Han Cao, and Leyi Xue. Research and application of knowledge graph in teaching: Take the database course as an example. *Journal of Physics: Conference Series*, 1607:012127, aug 2020. doi: 10.1088/1742-6596/1607/1/012127. URL `https://doi.org/10.1088/1742-6596/1607/1/012127`.

Shaoxiong Ji, Shirui Pan, Erik Cambria, Pekka Marttinen, and Philip S. Yu. A survey on knowledge graphs: Representation, acquisition and applications, 2020.

Apéndice A

Detalles de implementación

Este apéndice recoge los detalles de implementación del sistema de entrenamiento que no se consideraron parte de la contribución metodológica del Capítulo 4 pero que condicionan la reproducibilidad de los resultados y pueden resultar útiles para quien continúe el trabajo. Se describen el manejador de interrupciones y los mecanismos de recuperación del entrenamiento, el muestreo del *buffer* de repetición en PPO, el formato de los puntos de control, la estructura de *logging* y la convención de nomenclatura de los experimentos.

A.1. Gestión de interrupciones del entrenamiento

El bucle de entrenamiento se envuelve en el gestor de contexto `InterruptHandler`, que redirige dos señales del sistema operativo:

- `SIGINT` (**Ctrl**+**C**): el manejador guarda un *checkpoint* de emergencia con el estado del agente, cierra el *writer* de `TensorBoard` y termina el proceso de manera ordenada. El fichero de salida se nombra con marca de tiempo (`agent_interrupted_YYYY_MM_DD_HH_MM_SS.pth`) para no sobrescribir *checkpoints* anteriores. Además del estado de la red, se registra el paso global, el valor actual de ε (si el agente lo expone) y el tamaño del *buffer*.

- `SIGQUIT` (**Ctrl**+\): la señal es POSIX; en Windows la constante `SIGQUIT` no está definida y el gestor lo detecta y evita registrar el manejador. En sistemas POSIX, la señal lanza un *soft reset* que vacía el *buffer* de experiencia y restaura un valor alto de exploración ($\varepsilon \leftarrow 0{,}8$), diseñado para desatascar episodios en los que la política se ha estabilizado en un comportamiento improductivo. Esta funcionalidad se incluyó pensando en sesiones de depuración interactivas sobre Linux; dado que el equipo de desarrollo principal es Windows, no ha formado parte de los experimentos reportados.

Si durante el entrenamiento se produce una excepción no controlada (distinta de `KeyboardInterrupt`) el gestor registra un *checkpoint* de emergencia bajo el prefijo `agent_error_` y propaga la

excepción. En consecuencia, un fallo de software no destruye el progreso acumulado hasta ese momento.

A.2. Parada temprana por paciencia

El módulo `EarlyStopping` del paquete de entrenamiento monitoriza una métrica escalar (típicamente la recompensa acumulada media de los últimos episodios) y detiene el entrenamiento cuando la métrica no mejora durante `patience` episodios consecutivos. Al registrar la mejor puntuación, se guarda un *checkpoint* dedicado; de este modo, al terminar el entrenamiento, el usuario dispone del último modelo entrenado y del modelo con mejor rendimiento observado durante el proceso. Esta utilidad no se ha activado en los experimentos del Capítulo 5, cuyo horizonte estaba fijado a priori en 40 000 episodios.

A.3. Muestreo del *buffer* de experiencia en PPO

La particularidad del PPO empleado consiste en reutilizar el *buffer* de experiencia típico de los algoritmos *off-policy*. El flujo exacto es el siguiente:

1. Durante un episodio, el agente guarda en una memoria temporal las tuplas $(s_t, a_t, r_t, s_{t+1}, d_t, \log \pi_\theta(a_t \mid s_t), V_\phi(s_t))$ y una copia de la máscara de acciones válidas (si está habilitada).
2. Al terminar el episodio, se calculan las ventajas $\hat{A}_t$ y los retornos R_t mediante GAE con las recompensas y los valores del episodio recién concluido.
3. Cada tupla, extendida con $\hat{A}_t$ y R_t, se añade a un *buffer* circular de capacidad 10^5.
4. Si el *buffer* contiene al menos un mini-lote (tamaño 64), se muestrea un mini-lote aleatorio y se ejecuta un paso de optimización del objetivo PPO definido en la Sección 4.7.

En consecuencia, la actualización utiliza transiciones generadas por políticas antiguas. El objetivo recortado de PPO atenúa, pero no elimina, los efectos de esta discrepancia: el cociente de probabilidades $\rho_t(\theta)$ puede alejarse rápidamente del rango $[1-\varepsilon, 1+\varepsilon]$, lo que satura la pérdida y limita el tamaño efectivo del paso. Esta limitación se discute en la Sección 5.4.

A.4. Puntos de control y recuperación

Cada agente expone los métodos `save(path)` y `load(path)`, que serializan y restauran el estado completo (red o redes, parámetros del optimizador, paso global y, en el caso de DQN, el valor actual de ε). El directorio por defecto para los *checkpoints* de un experimento número `N` es `checkpoints/Test/N/<nombre_agente>/`, con tres tipos de fichero:

- `final.pth`: estado del agente al terminar el bucle.
- `best.pth` (si se activa la parada temprana): estado del agente con mejor métrica observada.
- `agent_interrupted_*.pth` o `agent_error_*.pth`: *checkpoints* de emergencia guardados por el `InterruptHandler` (Sección A.1).

El script `scripts/eval_agent.py` reconstruye el entorno a partir del fichero `config.json` escrito al inicio de la ejecución (o, en su defecto, a partir de las dimensiones del *checkpoint*) y aplica el enmascarado de acciones si la configuración original lo indicaba.

A.5. Métricas registradas en TensorBoard

El bucle de entrenamiento utiliza un `SummaryWriter` por agente y registra, por episodio, el conjunto homogéneo de métricas del Cuadro A.1. El uso de las mismas etiquetas para DQN y PPO permite comparar entre familias algorítmicas con los *scripts* auxiliares `scripts/plot_runs.py` y `scripts/compare_runs.py`. La configuración textual del agente y de la función de recompensa se guarda como *text summaries* bajo `Config/Agent` y `Config/Reward` al inicio de la ejecución.

Tabla A.1: Métricas registradas por el `BaseTrainer` en cada episodio de entrenamiento.

Etiqueta	**Significado**
`Reward/Total`	Recompensa acumulada del episodio.
`Reward/Avg`	Recompensa media por paso.
`Success/Episode`	1 si el agente alcanzó la meta, 0 en otro caso.
`Success/Map/<mapa>`	Serie específica por mapa de la métrica anterior.
`Done/Hit Obstacle`	1 si el episodio terminó en colisión.
`Done/Max Steps Reached`	1 si se agotó el horizonte temporal.
`Steps/Episode`	Número de pasos del episodio.
`Steps/Min/Episode`	Camino mínimo (BFS) desde el inicio hasta la meta.
`Steps/Ratio/Episode`	Cociente entre los dos anteriores.
`Distances/Start,End`	Distancia euclídea inicial y final a la meta.
`Loss/Episode`	Pérdida al final del episodio.
`Curriculum/Stage`	Etapa activa del currículo (si procede).
`Agent/*`	Métricas específicas del agente (p. ej. entropía).

A.6. Convención de nomenclatura de experimentos

Cada experimento se identifica por un número entero N y se materializa en tres artefactos paralelos:

- El *script* de entrenamiento `train/grid_training_NN.py`, que contiene la configuración específica del experimento. Los *scripts* de experimentos anteriores se conservan sin modificar como registro histórico; cuando la API cambia, el experimento siguiente se escribe en un fichero nuevo con el siguiente número.
- El directorio de logs `runs/Test/NN/<nombre_agente>/`, con los eventos de `TensorBoard` y el fichero `config.json` que serializa los argumentos de la línea de comandos, la configuración de la función de recompensa, los mapas utilizados y la configuración del LiDAR.
- El directorio de *checkpoints* `checkpoints/Test/NN/<nombre_agente>/`, que contiene los pesos entrenados.

Esta correspondencia uno a uno entre número de experimento, código, logs y pesos permite reproducir un resultado concreto a partir del `config.json` y del *checkpoint* `final.pth` del mismo directorio.

A.7. Entorno de ejecución y dependencias

El código se ha desarrollado en Python 3.11 y se instala en modo editable con `pip install -e .` desde la raíz del repositorio. Las dependencias principales son `gymnasium` (API del entorno), `torch` (redes neuronales y aceleración GPU), `tensorboard` (*logging*), `torchrl` (`EnvBase` auxiliar), `numpy`, `pygame` (visualización) y `tqdm` (barra de progreso). La selección del dispositivo de cómputo se realiza automáticamente al inicio de cada *script* con el patrón estándar `torch.device(çudaïf torch.cuda.is_available() else "mpsïf torch.backends.mps.is_available() else çpu")`, lo que permite ejecutar los mismos *scripts* en GPU NVIDIA, en Apple Silicon y en CPU sin modificaciones. La suite de pruebas unitarias (`pytest`, aproximadamente 113 tests sobre mapa, robot, entorno, recompensas, cabecera, entrenamiento y currículo) se ejecuta en unos 15 segundos y cubre los invariantes básicos de los componentes del simulador.

Apéndice B

Ejemplos en LaTeX

Este primer apéndice presenta ejemplos en LaTeXde cómo incluir referencias, citas bibliográficas, figuras, tablas o código. Este apéndice se deberá eliminar de la memoria antes de entregar el trabajo.

B.1. Referencias, citas y bibliografía

B.1.1. Referencias

Para poder referenciar un elemento dentro de la memoria hay que marcarlo con una etiqueta (`\label{[id]}`) que lo identifique inequívocamente. Es habitual utilizar identificadores representativos, por ejemplo, para marcar la introducción podemos utilizar una etiqueta como `\label{chap:introduccion}`

Una vez marcado el elemento (capítulo, sección, figura, tabla, ...) en LaTeX, utilizaremos el comando `ref` indicando a qué etiqueta queremos referenciar (`\ref{[id]}`). Por ejemplo, de esta manera podemos referenciar al capítulo de la introducción con 1. Podemos acompañarlo de un texto como "...como se vió en el capítulo 1..." o bien utilizar el comando `\autoref{[i]}`, que incluiría el tipo del elemento y aparecería en el texto como Capítulo 1, o incluso referenciarlo por el nombre del elemento con `\nameref{[id]}`, lo que se mostraría como Introducción.

B.1.2. Bibliografía

Para añadir una cita bibliográfica al documento tendremos que asegurarnos que en el fichero de bibliografía (bibliografía.bib) se encuentre la entrada bibliográfica correspondiente. Una vez que tengamos nuestra entrada bibliográfica utilizaremos el comando de cita de LaTeXindicando el identificador de la referencia bibliográfica, por ejemplo: `\cite{aikg}`, que se se visualizaría como Dessi et al. [2020].

Cuando queramos citar más de un trabajo en un mismo punto del documento se deberán

añadir a la cita separados por comas, por ejemplo: `\cite{chen_2018, Qin_2020}`, que se visualizaría como Chen et al. [2018], Qin et al. [2020]. En estos casos es habitual añadir las citas en orden cronológico de más antigua a más reciente.

Tal y como está configurada esta plantilla, por defecto el comando `\cite` realizará una cita textual (`\citet`), es decir, la cita forma parte del texto. Este tipo de citas se suelen realizar en frases como:

"Algunos ejemplos de grafos de conocimiento se presentan en Ji et al. [2020]"

Cuando la cita no forma parte del texto si no que se utiliza para reforzar una afirmación realizada en una frase, se debe utilizar una cita entre paréntesis (o corchetes). Para ello se usa el comando `\citep`, por ejemplo:

"Existe una gran variedad de aplicaciones de los grafos de conocimiento [Ji et al., 2020]."

B.2. Figuras

Para añadir una figura al documento utilizaremos el entorno figure, indicando la posición que debe tener dicha figura en el documento (h: here, t: top, b: botom; p: page), se recomienda en lo posible evitar de "!" que ignora todos los ajustes de los parámetros. El orden en el que se indiquen cada una de las opciones se tendrá en cuenta para colocar la figura, es decir si se indicase el orden [htbp] la figura primero se intentará colocar en el lugar que ocupa en el documento, si no se puede se intentará colocar al inicio (top) de la página, en caso que tampoco sea posible se intentará colocar al final (bottom) de la página y, por último en caso que no sea posible ninguna de las anteriores se colocará al inicio de una página nueva.



Figura B.1: Esta figura tiene una descripción al pie muy larga, por lo que añadiremos un título breve utilizando para ello los corchetes tras el comando \caption. La etiqueta de la figura (label) se incluirá al incio del flotante de la figura para que cualquier referencia cruzada (ref) a la misma lleve al inicio del flotante.

B.3. Tablas

Para añadir una tabla se utilizará el entorno `table`, indicando al incio de la tabla el título de la misma utilizando el `caption`. Un ejemplo puede verse en la Tabla B.1.

Tabla B.1: Esta tabla presenta un ejemplo con tres columnas y formato formal.

Model	Accuracy	Precision	Recall	F1-score	AUC
Modelo 1	0,33	0,75	0,72	0,42	0,21
Modelo 2	0,01	0,63	0,60	0,50	0,10
Modelo 3	0,03	0,93	0,33	0,04	0,42

B.4. Código

Para añadir código en la memoria utilizaremos el paquete `listing`, que permite mostrar código formateado en diversos lenguajes (Java, Python, C, ...).

```
import numpy as np

def incmatrix(genl1,genl2):
    m = len(genl1)
    n = len(genl2)
    M = None #to become the incidence matrix
    VT = np.zeros((n*m,1), int)  #dummy variable

    #compute the bitwise xor matrix
    M1 = bitxormatrix(genl1)
    M2 = np.triu(bitxormatrix(genl2),1)

    for i in range(m-1):
        for j in range(i+1, m):
            [r,c] = np.where(M2 == M1[i,j])
            for k in range(len(r)):
                VT[(i)*n + r[k]] = 1;
                VT[(i)*n + c[k]] = 1;
                VT[(j)*n + r[k]] = 1;
                VT[(j)*n + c[k]] = 1;

                if M is None:
                    M = np.copy(VT)
                else:
                    M = np.concatenate((M, VT), 1)

                VT = np.zeros((n*m,1), int)

    return M
```

B.5. Listas

Como en cualquier procesador de textos debemos diferenciar dos tipos de listas. Las listas no numeradas (o de viñetas) se definirán mediante entornos `itemize`.

- Elemento no numerado
- Elemento no numerado
- Elemento no numerado

Las listas numeradas se definirán mediantes entornos `enumerate`. En ambos casos, cada elemento de la lista se genera utilizando el comando `item`.

1. Elemento 1
   Con el comando \\ podemos insertar saltos de línea sin cambiar de párrafo.
2. Elemento 2. Con el comando `\textbf` podemos **enfatizar con negrita** un texto y con el comando `\textit` podemos *enfatizar con itálica* un texto.
3. Elemento 3

B.6. Ecuaciones

Cuando una ecuación va a ser referenciada desde el texto, en principio más de una vez, será necesario nombrar o numerar dicha ecuación (ver Ecuación B.1). Para añadir una ecuación numerada utilizaremos el entorno `equation`.

$$\left(\begin{array}{cccc} w_1, & w_2, & \ldots & , w_n \end{array} \right) \left(\begin{array}{c} x_1 \\ x_2 \\ \vdots \\ x_n \end{array} \right) + b = 0 \rightarrow \mathbf{w}^{\mathrm{t}}\mathbf{x} + b = 0 \tag{B.1}$$

Cuando la ecuación no va a ser referenciada desde el texto, podemos utilizar la versión de entorno no numerada `equation*`

$$\frac{1}{2} \parallel \mathbf{w} \parallel = \frac{1}{2}\sqrt{\sum w_i^2}$$

Es también posible añadir ecuaciones en línea con el texto, para ello se debe incluir la expresión matemática en LaTeXencerrada entre símbolos de dólar. Por ejemplo, la ecuación anterior se podría representar también en línea: $\frac{1}{2} \parallel \mathbf{w} \parallel = \frac{1}{2}\sqrt{\sum w_i^2}$.